One Char to Rule Them All: Systematically Exploring and Exploiting DNS Silent Vulnerabilities in Domain Name Resolution

Fasheng Miao[¶], Xiang Li[†✉], Changqing An[¶✉], Wenbin Xu[§], and Jilong Wang[¶‡]

[¶]*Institute for Network Sciences and Cyberspace, Tsinghua University*,
[†]*College of Cryptology and Cyber Science, Nankai University*,
[§]*School of Computer Science and Engineering, Beihang University*, [‡]*Quan Cheng Laboratory*

***Abstract*—Domain names function as human-readable identifiers on the Internet, with characters serving as their essential building blocks. However, since the initial specification of domain names in 1983, the security implications of handling special characters within the domain name resolution process have remained largely overlooked.**

In this work, we conducted the first systematic study of special character handling logic in DNS, reviewing DNS RFCs and analyzing 31 widely-used DNS software implementations through source code review and gray-box testing. Our systematic analysis reveals two new DNS logic vulnerabilities arising from inconsistencies and silent handling behaviors, leading to two classes of attacks (four variants) that affect all DNS roles, including stub resolvers, forwarders, recursive resolvers, and authoritative nameservers. We name them the SHAR attack. Attackers can exploit these vulnerabilities to launch DNS cache poisoning and load balancing disruption attacks. Through comprehensive experiments, we validate the impact on the real world. All 31 tested mainstream DNS software implementations are vulnerable to SHAR. Notably, attackers can seize control of domain names, even the entire TLD (e.g., `.ir`) or deceive victim resolvers to return invalid responses for legitimate queries, resulting in a persistent DoS effect. The SHAR attack can also enhance 10/13 well-known off-path DNS cache poisoning attacks (2002–2025). To further determine the impact in the wild, we test all DNS-related roles, including 21 mainstream Wi-Fi routers, 6 router OSes, 43 public DNS services, 883.5K stable open DNS resolvers, Root servers, TLD servers, SLD servers, and 223.6M domain names. The results show that the SHAR attack affects all tested Wi-Fi routers, router OSes, and public DNS services. In addition, we identify that 531.1K (60.1%) resolvers, 522 (36.1%) TLDs, and 12.5M (5.6%) domain names are also vulnerable to the SHAR attack. Following the best practice of responsible disclosure, we have reported these vulnerabilities to all affected vendors.

1. Introduction

The Domain Name System (DNS) translates human-readable domain names into machine-recognizable IP addresses through the resolution process [2], [3]. Web services, email delivery, content distribution, and user authentication all critically depend on the stability of domain name resolution [55], [88]. Consequently, attackers have developed various techniques to manipulate domain name resolution, including DNS cache poisoning [30], [31], [32], [33], [41], [54], [56], [60], [61], [79], [81], [82], [89] and denial of service (DoS) [14], [56], [66] attacks.

Domain names are fundamentally composed of a sequence of labels, each consisting of a series of characters [1], [2]. These characters serve as the DNS naming mechanism. In earlier specifications, domain labels were restricted to alphanumeric characters (`a-z, A-Z, 0-9`) and hyphens (-) [1], [3]. However, RFC 2181 states that implementations "*must not place any restrictions on label content*," implying that domain names can legally contain any character [23].

Prior work. In the area of DNS characters, previous studies primarily focused on homograph attacks [4], [26], [59], which exploit user confusion caused by Unicode characters (encoded in ASCII in DNS) to launch phishing attacks. Recent studies have also uncovered vulnerabilities arising from the ambiguities in the handling of "\" and `0x00` characters, which can be exploited to launch injection attacks [39], [40]. Prior work has primarily focused on the visibility of domain names or the handling of a very limited set of special characters. To date, *there have been no systematic studies that have attempted to understand all special characters handling logic throughout the domain name resolution process and explore potential logic vulnerabilities.*

Our study. To fill this research gap, we systematically analyzed all DNS roles by inspecting relevant RFCs and reverse engineering 31 widely used DNS software implementations through source code review and gray-box testing. Our in-depth analysis allowed us to summarize the general logic for handling special characters in domain names. Building on these insights, we crafted domain names with special characters to systematically test and compare the behavior of different implementations. This analysis revealed two previously unknown logic vulnerabilities exploitable for DNS cache poisoning and load-balancing disruption. *These vulnerabilities lie in the inconsistencies and silent behaviors in handling special characters, resulting in our proposed novel "*SHAR *attack"*. Notably, the SHAR attack targets all DNS roles using just a single special character, which inspired its name: "*One Char to Rule Them All*" (see Section 4).

2026 IEEE Symposium on Security and Privacy (SP) | 979-8-3315-6065-2/26/$31.00 ©2026 IEEE | DOI: 10.1109/SP63933.2026.00260

© 2026, Fasheng Miao. Under license to IEEE.

DOI 10.1109/SP63933.2026.00260
Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

Specifically, we observed that all DNS roles support special characters to some extent, but the supported character sets differ across software implementations. Queries containing unsupported characters are often silently dropped without generating any response. Furthermore, when upstream servers drop such queries without replying, downstream resolvers may remain indefinitely waiting, failing to respond to clients. These behaviors introduce the *silent* vulnerabilities, which could be exploited for cache poisoning (V_{EN} and V_{BF}) and load balancing disruption (V_{LB} and V_{SF}) attacks (see Section 5). These vulnerabilities affect all tested software. *(i)* V_{EN} can enhance previous DNS cache poisoning attacks by substantially extending the attack time window and bypassing the race with the responses from nameservers. *(ii)* V_{BF} revives the feasibility of brute-forcing source port and TxID, which has been widely believed infeasible since 2008. *(iii)* V_{LB} can disrupt the load balancing of recursive resolvers, concentrating DNS traffic on a single victim nameserver and weakening the overall resilience. *(iv)* V_{SF} induces resolvers to directly return invalid responses (`Servfail`), causing a DoS effect. *(ii)-(iv)* can attack arbitrary domains, even the entire TLDs, such as `.ir`, `.ai`, `.mil`, and `.info`.

We further investigated the real-world impact of SHAR attack in the wild. Our evaluation encompassed all DNS-related roles, including 21 Wi-Fi routers, 6 router OSes, 43 public DNS services, 883.5k open resolvers, Root servers, TLD servers, SLD servers, and 203.6M domain names. Through experiments with ethical considerations, we discovered that all Wi-Fi routers, router OSes, and public DNS services were vulnerable to the SHAR attack. Among public DNS services, 26 are vulnerable to V_{BF}, 25 are affected by V_{LB}, and 16 are affected by V_{SF}. Moreover, we discovered that 531.1K (60.1%) open resolvers, 522 (36.1%) TLDs, and 12.5M (5.6%) domain names are also susceptible to the SHAR attack. Notably, SHAR can enhance 10/13 previous DNS cache poisoning attacks. These results highlight the widespread, real-world impact of the novel attack.

In addition, we have reported discovered vulnerabilities to all the affected parties and are discussing solutions with them. Our work demonstrates that even the most basic characters may entail security risks and therefore warrant serious attention.

Contributions. We make the following contributions:

- We provided an in-depth analysis of special characters handling logic across all DNS roles.
- We discovered two novel logic vulnerabilities caused by the inconsistent and silent handling logic of special characters. Based on these vulnerabilities, we propose two classes of attacks (four variants), named SHAR attack, which can attack all DNS roles and enhance previous DNS cache poisoning attacks.
- We conducted comprehensive experiments to evaluate the real-world impact of the SHAR attack.
- We reported our newly discovered vulnerabilities to all affected parties, along with detailed mitigation and detection solutions.

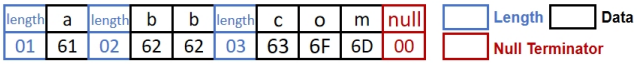


Figure 1: The DNS label encoding scheme (`a.bb.com`).

2. Background

In this section, we provide an overview of DNS characters and the domain name resolution process. Additionally, we systematically analyze DNS cache poisoning and load balancing disruption attacks.

2.1. DNS Characters

Domain names serve as human-readable identifiers on the Internet [1], [2]. These RFCs specify that each domain label may contain only alphanumeric characters (`a-z, A-Z, 0-9`) and hyphens (-), with hyphens disallowed at label boundaries [3]. In DNS packets, domain names use a length-value encoding [3], where each label is prefixed with a single-byte length field, followed by the label and terminated with `0x00`, as shown in Figure 1.

With the evolution of the Internet, the domain name character set has undergone significant changes. RFC 2181 [23] states that domain names should be treated as opaque binary labels at the protocol level, without character restrictions. While this improves protocol-level flexibility, it also introduces risks. For example, Jeitner et al. [39], [40] demonstrated that ambiguous handling of `0x00` and "\" characters can lead to injection attacks.

To support global usage, DNS introduced internationalized domain names (IDNs) [25], [49]. Although IDNs allow Unicode characters, they must be encoded into ASCII using Punycode for DNS transmission [19], [25]. IDNA2008 [24], [43], [48], [49], [50] further emphasized normalization and validity checks. However, *due to ambiguities in the standards and inconsistencies among implementations, the potential security risks in real-world Internet environments remain underexplored.*

In this work, we comprehensively examine the entire ASCII character set (`0x00-0x7F`), treating all non-alphanumeric characters (`a-z`, `A-Z`, `0-9`) as special characters. Detailed results on character support are summarized in Tables 10 and 11 (Appendix C).

2.2. Domain Name Resolution

As shown in Figure 2, the typical resolution process begins with the stub resolver, which is typically part of the client's OS [12]. This stub forwards the query to a local forwarder (home router or enterprise gateway), relaying the query to a recursive resolver (e.g., Google Public DNS, etc.). The recursive resolver traverses the DNS hierarchy, starting from root servers, to locate the nameserver responsible for the queried domain. Finally, the resource records (RR) hosted on the nameserver are returned through the resolution chain to the stub resolver [2], [3].

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

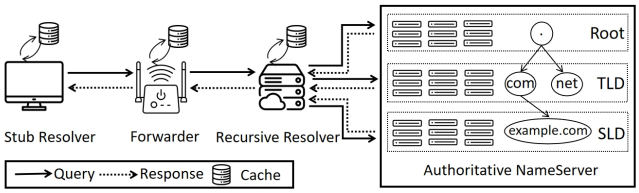


Figure 2: Domain name resolution.

This multistage resolution chain allows intermediate components to implement caching, filtering, or protocol normalization policies [2], [7]. However, *it also introduces potential vulnerabilities stemming from inconsistent handling or interpretation of special characters across different roles in the resolution process*.

2.3. DNS Cache Poisoning Attack

DNS cache poisoning [17] is a well-known attack that aims to inject forged RRs into the cache, thus manipulating the resolution of the targeted domains. The attackers can then redirect traffic and perform man-in-the-middle attacks [54], [88]. In this subsection, we survey DNS cache poisoning attacks (2002-2025) and identify potential vectors for enhancing existing attacks.

Basic attack. The core challenge of DNS cache poisoning lies in matching the source port and transaction ID (TxID). If an attacker can inject a forged response that matches both values *before* the legitimate reply arrives, the resolver will accept and cache it. The resolver will return crafted RRs for future queries. As shown in Figure 3, the steps of a DNS cache poisoning attack typically involve: *(i)* triggering a legitimate query; *(ii)* crafting spoofed responses; *(iii)* injecting them before the legitimate reply arrives; and *(iv)* poisoning the cache. To achieve *(iii)*, attackers typically try to attack from a neighboring host near the victim resolver (`Method1`) or reach the limit rate via DDoS attacks (`Method2`), thereby creating a time window to race against legitimate responses from the nameservers [54], [56]. However, techniques like Anycast routing and load balancing in modern networks make previous attacks increasingly impractical [56], [88].

The evolution of DNS cache poisoning attacks. Initially, attacks were *on-path*, where attackers intercepted and modified DNS packets to forge responses [18]. However, due to the impracticality for attackers to control the resolvers in practice [23], *off-path* attacks became mainstream. A pivotal milestone was Kaminsky's attack [42], which exploited the *birthday paradox* [79] by flooding random subdomain queries to increase the probability of correctly guessing the correct TxID within a limited window. This attack prompted source port randomization, which largely rendered brute-force guessing attacks infeasible [54], [56], [60], [61].

Since then, *off-path* attacks have employed advanced techniques, such as implementation vulnerabilities [20], [79], [82], protocol flaws [32], [81], [89], and side channels [56], [60], [61]. NAT port prediction [34] uses sequential UDP port allocation in NAT devices, and TxID prediction [46], [47] relies on weak generation algorithms.

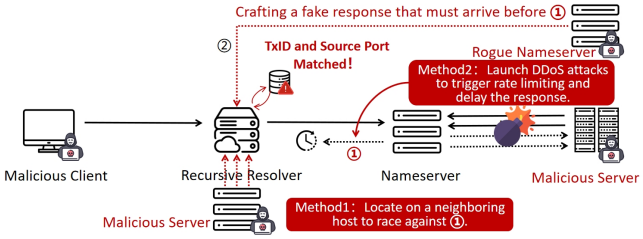


Figure 3: Basic DNS cache poisoning attack.

Additional techniques include ICMP side channels for port prediction [56], [60], downgrading DNSSEC validation [30], logic flaws in bailiwick checking [54], and response preprocessing vulnerabilities [56]. Jeitner [39], [40] exploited the parsing ambiguity of "\" and `0x00` to launch injection attacks.

Despite these advances, one critical attack factor is often overlooked: the crafted response must *arrive before* the legitimate reply from the nameserver. This timing constraint is fundamental to the feasibility of DNS cache poisoning in real-world environments. Furthermore, whether brute-force guessing of source ports and TxIDs can succeed within a practically feasible time remains an open question.

2.4. Load Balancing Disruption Attack

Load balancing is essential for enhancing the robustness and stability of DNS, as well as for reducing resolution latency [13]. To achieve low latency, recursive resolvers typically avoid querying unresponsive servers and prefer to query nameservers with better performance [68]. RFC 1034 also recommends that recursive resolvers should "*find the best servers to ask*" [2]. However, this optimization mechanism can be exploited by the load balancing disruption attacks. The attacker can bypass DoS defense strategies by concentrating traffic on specific victim servers, thereby compromising the availability of cloud service infrastructures that rely on DNS [88]. Prior studies have revealed design and implementation flaws in load balancing mechanisms across various application scenarios. Allman et al. [5] pointed out that certain configurations can undermine the intended robustness of the DNS ecosystem. Hao et al. [29] proposed a redirection hijacking attack that can bypass CDN load balancing and steer traffic toward a specific edge node. Su et al. [84] revealed that Akamai's load balancing strategies are vulnerable to intentional traffic manipulation, leading to service degradation or disruption. Pearce et al. [71] emphasized that load balancing algorithms should be carefully chosen according to the application context to avoid single points of failure and algorithmic misuse. Zhang et al. [88] exploited the vulnerability that nameservers do not respond to queries outside their DNS zones, combined with the observation that recursive resolvers often share nameserver preferences globally across domains. By deliberately assigning domain names to specific nameservers, they disrupted the load balancing strategy of recursive resolvers.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

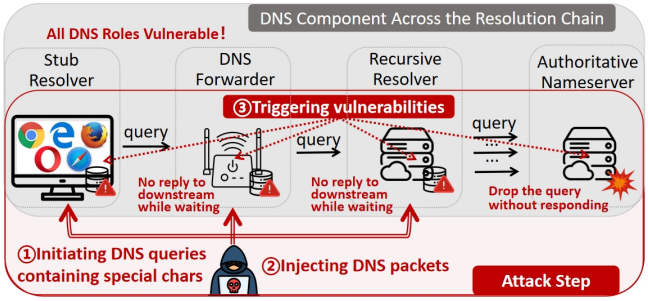


Figure 4: Threat model of the SHAR attack.

3. SHAR Attack Overview

In this paper, we uncover two novel logic vulnerabilities in handling special characters within the domain name resolution chain. Specifically, different roles involved in the chain may exhibit inconsistent support for certain special characters. For unsupported characters, they may directly discard the query without responding. When upstream servers remain *silent*, downstream resolvers will remain waiting, ultimately resulting in withheld responses to clients, causing them to exhibit similar *silent* behavior.

The *inconsistencies* and *silent* handling logic potentially introduce *silent* vulnerabilities, which can be exploited to launch DNS cache poisoning and load balancing disruption attacks using our newly proposed techniques. We name it SHAR attack (*Special Characters Attack*), which can target all DNS roles, such as stub resolvers, forwarders, and recursive resolvers. In this section, we first introduce the general threat model of SHAR attacks and then delve into the high-level workflow of SHAR attacks. Detailed exploitation techniques and experimental results are presented in Section 5.

3.1. Threat Model

The SHAR attack can target all DNS roles, including stub resolvers, forwarders, and recursive resolvers. In our threat model, we assume that the attacker has the following capabilities: *(i)* the ability to encode domain names into DNS packets and initiate DNS queries to target resolvers; *(ii)* the ability to observe the egress IP addresses of these resolvers. The threat model is shown in Figure 4.

The attacker can directly encode domain names containing special characters into DNS packets using the length-value encoding scheme [3], rather than relying on standard DNS libraries. For open resolvers accessible from the public Internet, the attacker can send DNS queries from arbitrary locations [80]. For resolvers restricted to specific networks, such as residential or enterprise environments, the attacker can leverage large-scale measurement platforms [76] or residential proxy networks [58], [64] to generate queries within the target networks. For stub resolvers, attackers can exploit online advertisements or spam emails embedding attacker-controlled domains to induce clients to initiate queries [56].

Before launching the SHAR attack, the attacker must first identify exploitable special characters and determine the egress IP address used by the resolver, to enable targeted packet injection. To this end, the attacker can send DNS queries containing various special characters to the target resolver and observe which characters are accepted and forwarded, while simultaneously recording the resolver's egress IP address as observed by an attacker-controlled nameserver. The attacker can register domain names through any standard domain registrar [27].

In the case of cache poisoning, the *off-path* attacker must spoof the source IP address of forged DNS responses. According to recent CAIDA data (May 2025) [15], more than 19.8% of IPv4 ASes are classified as IP-spoofable. This indicates that IP spoofing remains feasible, particularly via bulletproof hosting services [6] operating within these Autonomous Systems (ASes). We exclude resolvers implementing DNSSEC validation [77] or 0x20 encoding defenses [21]. For the load balancing disruption attack, the attacker needs to obtain the list of nameservers for the victim zone, which can be obtained through root or TLD nameservers [2], [3].

Notably, SHAR does not require the attacker to race legitimate responses by launching DDoS attacks against the nameserver or residing on a host located near the victim resolver. Instead, SHAR *can cause all DNS roles to remain silent by special characters, thereby introducing a novel and significantly enlarged attack surface*.

3.2. Attack Workflow

As shown in Figure 4, the SHAR attack begins with the attacker sending a DNS query containing special characters to the victim resolver (Step ①). This initial probe is designed to provoke the resolver into performing external queries with upstream servers or other stateful operations. This is followed by the injection of crafted packets (Step ②). Finally, once a role drops the crafted query without responding, the logical vulnerabilities are triggered (Step ③), causing the downstream components to remain in a state of persistent waiting. For load balancing disruption attacks, Step ② can be omitted because the silent behavior automatically reduces the resolver's preference for the affected nameserver, obviating the need to inject any forged packets. Based on this behavior, the attacker can launch the attack variants (named V_{EN}, V_{BF}, V_{LB} and V_{SF}) described below. For more detailed attack steps and specifics, please refer to Section 5.

Cache poisoning attack (V_{EN} and V_{BF}). The silence introduced by the SHAR attack significantly facilitates cache poisoning, including enhancing previous DNS cache poisoning techniques (V_{EN}) and making brute force of TxIDs and source ports feasible (V_{BF}) again.

V_{EN}: V_{EN} leverages the inconsistencies that resolvers can forward queries containing special characters while nameservers may directly discard them without a response, thereby opening a time window for the injection. The cache poisoning relies on the race that the attacker's crafted packets must arrive at the victim resolver before the legitimate answer, which is an inherently challenging task. To increase

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

the success rate, attackers often need to launch DDoS attacks to achieve the rate limit [60], [61] of the nameserver or reside on a neighboring host [54], [56] to race against the legitimate response. However, domains may deploy multiple nameservers, and the available time window is only 80ms-100ms [55] , which makes *off-path* attacks considered highly challenging in real-world networks. In contrast, V_{EN} enables the attacker to silence the nameserver with just a single special character, significantly extending the attack window (more than 60×)[1] and thus greatly advancing the cache poisoning attempt. Through a systematic analysis of attack steps and conditions, we find that V_{EN} can enhance 10/13 well-known cache poisoning attacks (2002–2025), including Kaminsky [41] and Tudoor [56] (see Section 5.1.1).

V_{BF}: V_{BF} leverages the significantly extended time window introduced by the silent behavior, making brute-force guessing of the source port and TxID feasible once again. The source port and TxID together constitute a 2^{32} guessing space. Brute force attacks have been considered infeasible due to the short response window (80ms–100ms) since 2008. However, V_{BF} can silence all DNS roles, effectively enlarging the attack time window and the attack surface. By leveraging the *birthday paradox* [79], the attacker can send enough queries per round to cover all 65,535 TxIDs while also guessing the source port. V_{BF} affects almost all widely used software and systems, such as BIND, Dnsmasq, and the Linux kernel. We conducted 20 rounds of attacks against mainstream software and all succeeded (see Section 5.1.2).

Load balancing disruption attack (V_{LB} and V_{SF}). By exploiting the recursive resolver's preference [2], [68], [88] in selecting nameservers and leveraging the silent vector introduced by SHAR, an attacker can disrupt load balancing mechanisms. This includes steering traffic toward a specific nameserver (V_{LB}) or returning invalid responses to all queries, thereby inducing DoS effects (V_{SF}).

V_{LB}: V_{LB} causes the nameservers to become silent, which in turn lowers their priority in the resolver's selection preference. The subsequent queries will be directed to higher-priority nameservers, thereby disrupting load balancing. The attacker can craft queries containing specific characters that only the victim supports (Respond to these crafted queries). This causes the victim to maintain a high priority, thereby concentrating subsequent traffic on it. V_{LB} affects widely used DNS software such as BIND, Unbound, Knot Resolver, Technitium, and 25 public DNS services, including AliDNS, Quad9 DNS, and Akamai Vantio DNS.

V_{SF}: V_{SF} forces all nameservers into silence, thereby lowering the priority in the resolver's selection preference. When subsequent queries target domains hosted on these nameservers, the resolver will directly return invalid responses without sending any queries. V_{SF} is a powerful attack, which can cause a persistent DoS effect on the entire TLD or SLD. V_{SF} affects Unbound, PowerDNS, and 16 public DNS services, such as Ali DNS, SafeDNS, and Level3 DNS. In addition, V_{SF} affects several famous TLDs, including `.ir` and `.mil`.

1. BIND and Unbound can provide a 10s attack window, MaraDNS and CoreDNS allow for a 6s window, and Dnsmasq offers over 10s [55].

4. Systematic Analysis of DNS Character Handling

Although upper-layer applications may include Unicode characters [25], [49], domain name handling and interactions in DNS are strictly based on ASCII [2], [3]. Notably, specifications for characters in domain names have evolved over time [1], [2], [23]. Different DNS roles may exhibit inconsistencies in support for certain characters, introducing potential security risks. Jeitner et al. [39], [40] demonstrated that ambiguities in handling "\" and `0x00` characters can be exploited to launch injection attacks. However, to date, no study has systematically analyzed how DNS handles the full set of special characters, particularly the inconsistencies across different roles.

Table 1 lists the DNS software analyzed in this study, covering all DNS resolver roles and mainstream implementations (latest versions): 8 recursive resolvers, 13 DNS forwarders, 6 stub resolvers, and 4 DNS libraries. Considering DNS software's complex and inconsistent implementations, we *reverse engineered* all resolvers, captured incoming and outgoing packets, and monitored runtime behaviors and logs across 5 rounds to identify vulnerabilities. Furthermore, we conducted source code analysis to validate our findings. The systematic analysis of characters supported by each resolver took 6 weeks. In summary, we consolidated the generic workflow and identified newly discovered logical vulnerabilities that attackers could exploit to launch various attacks, such as DNS cache poisoning and load balancing disruption. These attacks can target all DNS roles.

4.1. Generic Workflow of DNS Character Handling

Through in-depth analysis, including RFC documentation, source code review, single-step debugging, and gray-box testing, we have found that all DNS roles perform domain name validation. We summarize the generic workflow of DNS character handling as follows. A detailed process can be found in Algorithm 1 from Appendix A.

(i) WireFormatParsing. This step parses the raw DNS packet, including decoding domain names, verifying label lengths ($\leq$ 63 bytes), domain name length ($\leq$ 255 bytes), and that compression pointers are valid and loop-free [3].

(ii) BasicFormatValidation. This step validates the overall structure of the DNS message. It checks that the header fields are consistent, all sections (Question, Answer, Authority, Additional) match their declared counts, and each domain name is properly terminated with `0x00` [3]. These checks ensure the packet is syntactically well-formed [2].

(iii) SemanticValidation. Each domain name is validated against character-level rules defined by RFCs, such as allowable character sets and label composition constraints.

(iv) PolicyValidation. Names are further checked against implementation-specific or configuration-defined policies,

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

such as `check-names` [9], which may enforce stricter rules depending on the record type and operational settings.

If a domain name fails validation at any of the above steps, the DNS packet will be dropped. We separately tested encoding the dot character "." `(0x2E)` as a single label in the domain name and found that all implementations discard the DNS packets. Therefore, in subsequent discussions, we exclude this special character from our consideration.

4.2. Software Implementations of Special Character Handling

We summarize the implementation differences in DNS character handling logic across the analyzed software and illustrate their specific behaviors in Table 1. For special characters, 25 resolvers support all. Additionally, when the upstream server remains silent, 21 resolvers will enter a waiting state and not respond to the downstream server. This behavior exposes a significant attack surface for launching SHAR attacks.

Analysis Methodology. We *reviewed the source code* of all DNS software and focused on their character handling modules through line-by-line source code inspection and GDB debugging to ensure completeness. For closed-source software such as Microsoft DNS [65] and macOS, we relied on official documentation and conducted empirical testing using crafted packets inspired by the behavior of open-source implementations. To facilitate our measurements, we registered the domain `attacker.com` and hosted it on our controlled nameserver. We constructed subdomains in the format `{char}.{nonce}.attacker.com`, directly encoding them into DNS query packets, where `char` is a set that contains all the characters we intend to test and `nonce` is a random string to bypass caching. We capture all incoming and outgoing packets.

Recursive resolvers. Except for Technitium DNS and MaraDNS, all resolvers support all special characters.

- **BIND** (`dns_name_fromwire()` [10]), **Microsoft DNS**, **Unbound** (`dname_valid()` [86]) and **Simple DNS Plus** support all characters. If upstream servers remain silent, they will not respond to downstream servers.
- **Knot Resolver** (`knot_dname_from_str()` [51]) and **PowerDNS** (`packetParser()` [74]) support all characters. If upstream nameservers remain silent, they will return a `ServFail` response.
- **Technitium** (`ConvertToByteKey()` [85]) only supports the `*` character. If the domain name contains unsupported characters or upstream nameservers remain silent, it returns a `ServFail` response.
- **MaraDNS** (`decomp_get_label()` [62]) only supports printable characters. If the domain name contains non-printable characters (`0x00-0x1F and 0x7F`) or upstream servers keep silent, it will not respond to downstream servers.

DNS forwarders. Except for SmartDNS, all DNS forwarders support all special characters.

- **Dnsmasq** (`extract_name()` [22]), **DNSDist**, **Pi-hole**(`extract_name()` [72]), **Acrylic DNS**, **AdGuard**, **AdGuard Home**, **Nxfilter**, **Dual DHCP DNS** and **YogaDNS** can forward all characters. If upstream resolvers remain silent, they will not respond to downstream servers.
- **CoreDNS**, **DNS Safety** and **pdnsd** can forward all characters. If upstream resolvers remain silent, it will return a `ServFail` response.
- **SmartDNS** can forward all characters except for `NULL (0x00)` and the hyphen "-" `(0x2D)`. If domain names contain `NULL` and "-", or if upstream resolvers keep silent, it will return a `ServFail` response.

Stub resolvers. Except for ChromeOS, which can only forward "_" `(0x5F)`, all stub resolvers support all special characters.

- **Linux** can forward all characters. If upstream resolvers remain silent, it will return an `NXDomain` response.
- **MacOS, iOS, Windows, and Android** can forward all characters, and if upstream resolvers remain silent, it will not return any response to downstream servers.
- **ChromeOS** (`isValid()` [16]) only forward "_" `(0x5F)` special character. If upstream resolvers remain silent, it will not respond to downstream servers.

DNS libraries. Except for JavaScript, all DNS libraries support all special characters.

- **Python** uses the `dnspython` library to process DNS packets. If upstream resolvers remain silent, `dnspython` will not respond.
- **Golang** uses the `github.com/miekg/dns` [28] library to process DNS packets. If upstream resolvers remain silent, it will not respond.
- **JavaScript** uses the `node:dns/promises` library to process DNS packets and can only forward "*" `(0x2A)`, "–" `(0x2D)`, "/" `(0x2F)`, "\" `(0x5C)`, and "_" `(0x5F)` characters. If domain names contain unsupported characters, it will return a `query A EBADNAME domain`. If upstream resolvers remain silent, it will not respond.
- **Java** uses the `dnsjava` library to process DNS packets. If upstream resolvers remain silent, it returns a `network error`, and the next two queries are dropped, which will be repeated.

Nameservers. Based on the best free DNS server list [57], we evaluated 7 mainstream nameserver implementations using their latest versions, including BIND, PowerDNS, NSD, Technitium, Knot DNS, gdnsd, and MaraDNS. For all special characters, all software will respond to the queries, with detailed behavior summarized as follows:

- **BIND, NSD, Knot DNS, gdnsd, and MaraDNS** will return an `NXDomain` response for all tested queries.
- **PowerDNS** will return an `NXDomain` response if queries contain " " `(0x20)`, "-" `(0x2D)`, "/" `(0x2F)`, ":" `(0x3A)`,"@" `(0x40)`, "\" `(0x5C)`, and "_" `(0x5F)` characters. For other special characters, it will return a `Servfail` response.
- **Technitium** will return an `NXDomain` response if queries contain "*" `(0x2A)`, "/" `(0x2F)`, and "_" `(0x5F)` characters. For other special characters, it will return a `Formerr` response.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

TABLE 1: Special characters handling implementations of 31 DNS resolver software (All Vulnerable)

Resolver				Characters		Response Result			Vulnerability		
Role	Index	Software	Version	Unsupport	Support	Unsupport Response	Support Response	Support Status	V_{BF}	V_{LB}	V_{SF}
	# 1	BIND	9.18.30	0	65 (All)	–	✗	–	✓	✓	✗
	# 2	Unbound	1.19.2	0	65 (All)	–	✗	–	✗	✓	✓
	# 3	Knot	5.7.3	0	65 (All)	–	✓	Servfail	✗	✓	✗
Recursive	# 4	PowerDNS	4.9.3	0	65 (All)	–	✓	Servfail	✗	✓	✓
Resolver	# 5	Microsoft	2024	0	65 (All)	–	✗	–	✓	✗	✗
	# 6	Simple DNS+	9.1	0	65 (All)	–	✗	–	✓	✗	✗
	# 7	Technitium	13.4.3	62	3	✓	✓	Servfail	✓	✓	✗
	# 8	MaraDNS	3.5.0036	33	32	✗	✗	–	✓	✗	✗
	# 9	Dnsmasq	2.90	0	65 (All)	–	✗	–	✓	N/A	N/A
	# 10	CoreDNS	1.12.0	0	65 (All)	–	✓	Servfail	✓	N/A	N/A
	# 11	DNSDist	1.4.0	0	65 (All)	–	✗	–	✓†*	N/A	N/A
	# 12	SmartDNS	45	2	63	✓	✓	Servfail	✓	N/A	N/A
	# 13	Pi-hole	6.1	0	65 (All)	–	✗	–	✓*	N/A	N/A
	# 14	pdnsd	1.2.9	0	65 (All)	–	✓	Servfail	✓*	N/A	N/A
Forwarder	# 15	Acrylic DNS	2.2.1	0	65 (All)	–	✗	–	✓*	N/A	N/A
	# 16	AdGuard	7.20.0	0	65 (All)	–	✗	–	✓*	N/A	N/A
	# 17	AdGuard Home	0.107.57	0	65 (All)	–	✗	–	✓*	N/A	N/A
	# 18	DNS Safety	2.1.0	0	65 (All)	–	✓	Servfail	✓*	N/A	N/A
	# 19	Dual DHCP DNS	8.00RC	0	65 (All)	–	✗	–	✓*	N/A	N/A
	# 20	Nxfilter	4.7.1.7	0	65 (All)	–	✗	–	✓*	N/A	N/A
	# 21	YogaDNS	1.45	0	65 (All)	–	✗	–	✓*	N/A	N/A
	# 22	Linux	249	0	65 (All)	–	✓	NXDomain	✓	N/A	N/A
	# 23	Windows	2025	0	65 (All)	–	✗	–	✓	N/A	N/A
Stub	# 24	MacOS	15.2	0	65 (All)	–	✗	–	✓	N/A	N/A
Resolver	# 25	iOS	18.2	0	65 (All)	–	✗	–	✓	N/A	N/A
	# 26	Android	15	0	65 (All)	–	✗	–	✓	N/A	N/A
	# 27	ChromeOS	126.0.xxx	64	1	✗	✗	–	✓	N/A	N/A
	# 28	Python	2.4.2	0	65 (All)	–	✗	–	✓*	N/A	N/A
DNS	# 29	Golang	v1.1.65	0	65 (All)	–	✗	–	✓*	N/A	N/A
Library	# 30	JavaScript	v22.15.0	60	5	✓	✗	–	✓*	N/A	N/A
	# 31	Java	3.5.2	0	65 (All)	–	✓	Network Error	✓*	N/A	✓

"-": Applicable due to no unsupported characters or no response. ✓: Yes. ✗: No. ✓: Vulnerable. ✗: Not vulnerable.
"*": Poor randomization of source port. "†": Poor randomization of TxID

4.3. Novel Vulnerabilities in Character Handling Logic

Through a comprehensive analysis of character-handling logic in 31 DNS software implementations, we reveal inconsistencies and uncover several novel logic vulnerabilities affecting all tested systems (see the "Vulnerability" column in Table 1). These vulnerabilities can be divided into two categories based on their impact: cache poisoning (V_{BF} in 28 software) and load balancing disruption (V_{LB} in 5 software, V_{SF} in 3 software). Remarkably, the V_{BF} vulnerability illustrates the fundamental insecurity of the DNS protocol, as it can be exploited without advanced techniques. This section presents a high-level analysis of the root causes and exploitation potential of these vulnerabilities. The specific attack vectors and steps are discussed in detail in Section 5.

DNS cache poisoning (V_{BF} and V_{EN}): We observe inconsistent handling of special characters across different DNS software. MaraDNS and ChromeOS directly discard DNS queries containing unsupported characters. Additionally, 22 software remain waiting if upstream servers discard the queries. This vulnerability exposes downstream DNS roles to cache poisoning via V_{BF}. We further identify poor source port randomization in DNSDist, Pi-hole, pdnsd, Acrylic DNS, AdGuard, AdGuard Home, DNS Safety, Dual DHCP DNS, NxFilter, YogaDNS, and Python, Go, JavaScript, Java libraries. DNSDist also exhibits poor TxID randomization. These vulnerabilities significantly facilitate V_{BF}. The above vulnerabilities also enhance previous cache poisoning attacks by bypassing the race with the nameserver. V_{EN} can enhance 10/13 previous attacks (see Section 5.1.1).

Load balancing disruption (V_{LB} and V_{SF}): The nameserver may directly drop the query containing unsupported characters without any response. The priority of these nameservers will be lowered, and resolvers typically exhibit a stronger preference for higher-priority nameservers. Subsequent queries will be redirected to other nameservers. This vulnerability enables the attacker to disrupt load balancing and concentrate traffic on a victim nameserver. V_{LB} can affect BIND, Unbound, Knot, PowerDNS, and Technitium. Furthermore, when all upstream nameservers drop the query, Unbound and PowerDNS will mark them as unreachable, and subsequent queries will be responded with invalid responses (`ServFail`). In addition, we observe that `dnsjava` drops the next two queries following the no response from upstream servers, returning a `network error` response. The attacker can exploit the V_{SF} to trigger a DOS attack, affecting Unbound, PowerDNS, and `dnsjava`.

5. Two SHAR Attacks (Four Variants)

DNS caching [2] and load balancing [13] are two fundamental mechanisms for enhancing the performance and robustness of domain name resolution [3]. In this section,

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

TABLE 2: Well-known DNS cache poisoning attacks (2002–2025): 10/13 can be enhanced by V_{EN}.

Cache Poisoning		Attack Impact			Can Be
Work	Year	Stub	Forwarder	Recursive	Enhanced
[79]	2002	✗	✗	✓	✓
[41]	2008	✗	✗	✓	✓
[82]	2010	✗	✗	✓	✓
[31]	2012	✗	✗	✓	✓
[32]	2013	✗	✓	✓	✗
[33]	2013	✓	✗	✓	✓
[81]	2014	✓	✗	✓	✗
[89]	2020	✗	✓	✗	✗
[60]	2020	✓	✓	✓	✓
[61]	2021	✓	✓	✓	✓
[30]	2023	✓	✓	✓	✓
[54]	2023	✗	✗	✓	✓
[56]	2024	✓	✓	✓	✓

✓: Can target it. ✗: Can't target it. ✓: Enhanced. ✗: Not enhanced.

we present detailed exploitation procedures for the two proposed SHAR attacks (Four variants), end-to-end experiments conducted under controlled conditions, along with a comparison with prior attacks. The results show that SHAR can effectively compromise both caching and load balancing mechanisms by seizing control of domains, even including TLDs, such as `.ir`, `.info`, and `.ai`, or deceiving victim resolvers into returning invalid responses for legitimate queries, resulting in a DoS effect.

Experiment setup. We conducted the cache poisoning (V_{BF}) and load balancing disruption attacks (V_{LB} and V_{SF}) using programs developed in Go, which implement the techniques below. We deployed Dnsmasq, BIND, Unbound, Knot, PowerDNS, and Technitium on separate Ubuntu 24.04 servers, while Microsoft DNS, MaraDNS, and Simple DNS+ were installed on separate Windows Server 2025 machines. Each domain was configured with 4 nameservers. The attack program and all servers were connected via a local network (500 Mbps bandwidth) and assigned public IP addresses. For V_{EN}, we systematically revisited all previous DNS cache poisoning attacks (2002-2025), analyzing how V_{EN} enhances them, as shown in Table 2.

5.1. DNS Cache Poisoning Attack

DNS cache poisoning is a powerful attack, as successfully injecting forged DNS responses can compromise all clients relying on the affected resolver. Currently, state-of-the-art attackers often employ advanced techniques to bypass the 2^{32} (TxID $\times$ Source port) guessing space, such as protocol flaws [32], [81], [89] and side channels [56], [60], [61]. However, the above DNS cache poisoning attacks still fundamentally depend on the forged response reaching the resolver before the legitimate reply (80ms-100ms) [55].

We demonstrate that the proposed SHAR attack enables a novel attack vector that bypasses the race condition with legitimate replies and significantly improves the practicality of previous DNS cache poisoning. The key insight of SHAR lies in exploiting inconsistencies in the handling of special characters across different DNS roles, which can lead to silent vulnerabilities and significantly extend the attack time window. SHAR introduces two attack variants: V_{EN} and V_{BF}.

TABLE 3: The experimental results of the V_{BF} attack.

Resolver		Avg. Round	Avg. Time	Success
Role	Software	Taken	Taken (s)	Rate
Stub	Linux	7,532	8,687	20/20
Forwarder	Dnsmasq	7,947	9,469	20/20
Recursive	BIND	7,282	9,717	20/20

5.1.1. V_{EN}: Enhancing previous DNS cache poisoning attacks. In cache poisoning attacks, spoofed responses from the attacker must reach the target resolver before the legitimate responses. Consequently, *off-path* cache poisoning attacks are generally difficult to exploit at scale. We observe that MaraDNS and Chrome OS directly discard queries containing unsupported characters. Moreover, when upstream nameservers drop such queries, 22 resolvers remain indefinitely waiting and fail to respond to downstream resolvers. As shown in Figure 5(a), SHAR enhances the threat model by bypassing the race against legitimate responses. Without modifying attack steps, V_{EN} injects special characters into the domain name, causing the target resolver to remain silent and allowing cache poisoning.

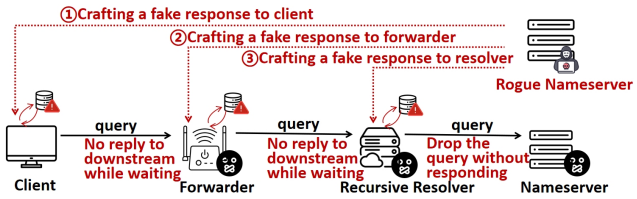
Attack design. As shown in Table 2, we systematically reviewed all *off-path* cache poisoning attacks from 2002 to 2025, as attacks before 2002 were mainly *on-path* and therefore excluded. We conducted a systematic analysis of the attack steps and vectors for all previous attacks, reproducing them using domains and machines under our control. Our analysis reveals that attacks leveraging IP fragmentation do not require a race condition, as forged fragments can persist for up to 30s in the buffer [32], [81], [89]. In contrast, all other DNS cache poisoning techniques depend on racing against the nameserver. V_{EN} can advance 10/13 cache poisoning attacks, including Kaminsky [41] and Tudoor [56], and is effective against all DNS resolver roles.

5.1.2. V_{BF}: Brute-forcing TxID and source port with high feasibility. The DNS protocol relies on a 16-bit TxID and a 16-bit source port to uniquely identify each query, resulting in a 2^{32} guessing space. Brute-forcing is typically considered impractical, as attackers generally have only a narrow time window (80–100ms) to attempt it. V_{BF} fundamentally improves brute-forcing model via a special character. V_{BF} induces silence from upstream resolvers or nameservers, instead of triggering invalid responses such as `NXDomain` or `ServFail`. This behavior causes resolvers to remain waiting without timely termination (e.g., 5s) [56], enabling attackers to brute-force spoofed responses and poison stub resolvers, forwarders, and recursive resolvers. V_{BF} affects 26 public DNS services.

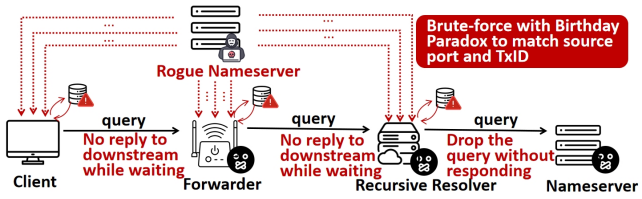
In addition, we identify poor source port randomization in DNSDist, Pi-hole, pdnsd, Acrylic DNS, AdGuard, AdGuard Home, DNS Safety, Dual DHCP DNS, NxFilter, YogaDNS, and the DNS libraries of Python, Go, JavaScript, and Java. DNSDist also exhibits poor TxID randomization. Such poor randomization can further facilitate V_{BF} attacks. Moreover, the success rate of V_{BF} can be further enhanced using the *Birthday Paradox* [79], [82].

Attack design. As shown in Figure 5(b), V_{BF} attack pro-

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.



(a) The threat model of V_{EN}.



(b) The threat model of V_{BF}.

Figure 5: The threat model of DNS cache poisoning attack. (a) V_{EN}, (b) V_{BF}.

ceeds as the following: *(i)* the attacker sends a DNS query in the form of {char}.{nonce}.attacker.com, where nonce is a random string to bypass cache, and char is crafted to trigger silent vulnerabilities; *(ii)* the resolver forwards the query to the upstream nameserver, assigning a random TxID and source port; *(iii)* If a nameserver silently drops the crafted packet, the downstream resolver will remain in a prolonged waiting state. Within this extended time window, the attacker can inject spoofed responses in an attempt to guess the correct TxID and port. In our experiment, we probe 5 ports and brute-force all 65,536 TxIDs per round; *(iv)* the attacker checks if the returned IP matches a sentinel (e.g., a.t.k.r) for success. If it resolves to a fallback (e.g., v.c.t.m), the attack fails and a new round begins.

Based on the *Birthday Paradox*, the probability of a successful attack increases with the number of guessing rounds (each taking 1s). The success probability after x rounds is given by $P_{\text{success}} = 1 - \left(1 - \frac{5}{65{,}536}\right)^x$, where x is the number of rounds and 5 is the number of source ports guessed per round. As x increases, the probability approaches 1, ensuring V_{BF} can be successful.

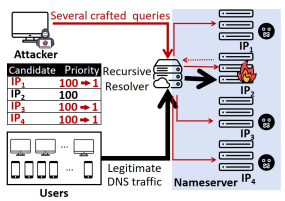
End-to-end attacks. V_{BF} can poison all roles in the DNS resolvers via just a single character. We selected Linux (stub resolver), Dnsmasq (forwarder), and BIND (recursive resolver) as our attack targets based on their popularity [83].

As shown in Table 3, we repeated the attack 20 times. The results demonstrate that V_{BF} makes cache poisoning feasible by brute-forcing the TxID and source port within a reasonable time, which was considered impractical since 2008. Specifically, the shortest successful poisoning time observed was 193s for Dnsmasq, 228s for Linux, and 971s for BIND. The longest recorded times were 21,903s for Dnsmasq, 20,100s for Linux, and 20,983s for BIND. During the attack process, the peak bandwidth is 297 Mbps.

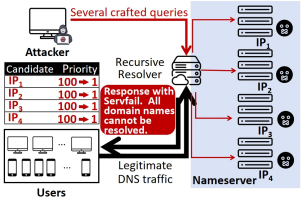
While V_{BF} requires more time (average 2.5 hours) than attacks leveraging other advanced techniques (average 500s) [54], [56], [60], [61], it fundamentally demonstrates that the DNS remains insecure. Moreover, this cost is entirely acceptable, as the attack impacts all subsequent client queries. By controlling the injected TTL value, the attacker can maintain the effect for months or even longer [52].

5.2. Load Balancing Disruption Attack

The load balancing disruption attack is highly effective in bypassing DoS defenses, overloading nameservers,



(a) The threat model of V_{LB}.



(b) The threat model of V_{SF}.

Figure 6: The threat model of DNS load balancing disruption attack. (a) V_{LB}, (b) V_{SF}.

lowering the barrier for traffic hijacking, and DNS cache poisoning [88]. SHAR can effectively achieve this using only a single special character, introducing two attack variants: V_{LB} and V_{SF}. As shown in Figure 6, V_{LB} can concentrate DNS traffic on a single victim nameserver, thus overloading it and weakening the overall DNS resilience. More seriously, V_{SF} can deceive the victim resolver, directly returning invalid responses (Servfail) to clients without querying the nameserver.

V_{LB}: **Concentrating DNS traffic on a single victim nameserver.** Resolvers exhibit a strong preference for well-performing nameservers [2], [68], [88]. Attackers can exploit this preference to disrupt load balancing mechanism. Specifically, if a nameserver fails to respond, the resolver will downgrade its priority and avoid sending further queries to it. As shown in Figure 6(a), the attacker crafts queries containing specific characters supported by only one nameserver, while others discard them. Consequently, only the responsive nameserver maintains a high priority, causing subsequent queries to be concentrated on it.

V_{SF}: **Inducing resolvers to directly return invalid responses.** To reduce resource consumption, resolvers typically send queries only to operational nameservers [2], [56], [88]. Attackers can exploit this mechanism to launch DoS attacks. As shown in Figure 6(b), the attacker crafts queries containing specific characters that are discarded by all nameservers. Consequently, the resolver marks all nameservers as faulty. For subsequent queries, if all nameservers for the domain are considered faulty, the resolver will directly return invalid responses (Servfail) without attempting further queries. This technique effectively renders the domain unresolvable and forcibly breaks the resolution chain.

Attack design. The attack steps for V_{LB} and V_{SF} are shown in Figure 6. *(i)* The attacker sends multiple crafted DNS

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

queries in the format {char}.{nonce}.attacker.com, where nonce bypasses caching and char triggers silent vulnerabilities. *(ii)* The resolver forwards queries to all upstream nameservers. To ensure each nameserver receives at least one query, the attacker simply sends multiple queries. *(iii)* If a nameserver does not support the char, it silently discards the query, and the resolver correspondingly lowers that nameserver's priority. If only one nameserver supports it, then only that nameserver retains a high priority, thereby triggering V_{LB}, under which all queries are directed to it. If none of the nameservers support it, the priority of all nameservers is reduced, and V_{SF} is triggered. *(iv)* As shown in Figure 6(a), under V_{LB}, future queries are concentrated on a single nameserver, overloading it. As shown in Figure 6(b), under V_{SF}, the resolver directly returns ServFail for future queries, causing a DoS effect.

End-to-end attacks. Only recursive resolvers support load balancing mechanisms. We selected 8 mainstream recursive resolvers as our targets. The results show that V_{LB} affects 5 resolvers, including BIND, Unbound, Knot, PowerDNS, and Technitium. V_{SF} affected Unbound and PowerDNS. PowerDNS and Unbound exhibit a strong preference for well-performing nameservers and avoid querying those with low priority, even when all nameservers are of low priority, making them susceptible to both V_{LB} and V_{SF}.

As shown in Table 4, we further evaluated the scope and duration of the attack, covering SLDs, TLDs, or all domains hosted on the targeted nameservers. For V_{LB}, BIND, Knot, PowerDNS, and Technitium were found to impact all domains hosted on the victim nameservers. In all affected resolvers, the disruption lasted more than 36 hours. For V_{SF}, Unbound and PowerDNS exhibit divergent behaviors. Compared to state-of-the-art attacks [88], our proposed load balancing disruption attack can target the entire TLD and cause a DoS effect, posing a greater real-world impact.

Unbound. If the nameserver does not respond, the resolver will temporarily stop sending queries to that nameserver for 900s. Therefore, the impact of V_{SF} can persist for 900s. For domains within different SLDs, the resolver will send queries to the nameserver. Therefore, the attacker can send a query to Unbound every 900s to keep V_{SF} active, effectively causing a persistent DoS for the entire SLD.

PowerDNS. If the nameserver fails to respond 60 consecutive times, PowerDNS will stop trying it. After the 60s, it attempts to send another query. If there is still no response, it waits another 60s. During this period, PowerDNS avoids querying the nameserver for all domains hosted on it. Therefore, the attacker can send a single query to PowerDNS every 60 seconds to keep V_{SF} active, leading to a persistent DoS for all domains hosted on the vulnerable nameserver.

5.3. Discussion

For practical attacks, we need to consider the following factors that affect the attack result and the actual attack surface in the real world.

Multiple nameservers. Domain names may be configured with multiple nameserver IPs, which traditionally complicates the attack [54], [56], [60], [61], [89]. However, since SHAR can force nameservers into silence, multiple nameservers do not reduce the effectiveness of SHAR. Domains with more nameservers often indicate higher importance or a larger number of hosted domains [5], [69], thereby amplifying the impact of the SHAR attack.

TABLE 4: The experimental results of V_{LB} and V_{SF} attacks.

Recursive Resolver	V_{LB} Impact				V_{SF} Impact			
	SLD	TLD	ALL	Time	SLD	TLD	ALL	Time
BIND	✓	✓	✓	∞	✗	✗	✗	–
Unbound	✓	✗	✗	∞	✓	✗	✗	900s
Knot	✓	✓	✓	∞	✗	✗	✗	–
PowerDNS	✓	✓	✓	∞	✓	✓	✓	60s
Microsoft	✗	✗	✗	–	✗	✗	✗	–
Simple DNS+	✗	✗	✗	–	✗	✗	✗	–
Technitium	✓	✓	✓	∞	✗	✗	✗	–
MaraDNS	✗	✗	✗	–	✗	✗	✗	–

✓: Vulnerable. ✗: Not vulnerable.
∞: The impact of the vulnerability lasted more than 36 hours.

Multiple backend IPs. Public DNS services are typically backed by multiple IP addresses, which theoretically expands the injection space for attackers. However, backend IP selection is often constrained by factors such as geolocation and load balancing policies. However, affected public DNS services usually rely on a small set of backend IPs (e.g., 1–4) from a given location [55], [56]. This significantly lowers the attack complexity, as the adversary only needs to spoof a limited number of IPs (V_{EN} and V_{BF}) or just send more queries to disrupt the load balance (V_{LB} and V_{SF}).

Impact of QNAME minimization. As shown in Section 6.4, the SHAR attack can affect 36.1% of TLDs, including .info, .ai, .io, and .ir. It enables the attacker to poison NS queries issued by recursive resolvers, thereby seizing the entire SLD zone. V_{LB} and V_{SF} can be leveraged to disrupt load balancing, and even cause a persistent DoS effect on the entire TLD. However, if the resolver supports query name minimization (*qmin*) [11], the effectiveness of the SHAR attack is reduced, as the encoded special characters are filtered out when querying the TLD. Nevertheless, it can still target SLDs, since queries such as !.example.com will still be sent to the example.com nameservers, and the nameserver will be silent without response due to the special character. For cache poisoning attacks, the attacker can send an empty answer section along with NS records in the authority section that contain glue-poisoning RRs for ns1.example.com or other RRs, thereby affecting all domains that rely on these NS records. For load balancing disruption attacks, the resolver will reduce the priority of the silent nameserver, consequently impacting all domains hosted on that nameserver. Therefore, support for *qmin* does not entirely mitigate SHAR attack.

Attack the domains. The SHAR attack inherently targets valid domain names, including entire TLD zones, by exploiting special characters. In the case of cache poisoning, an attacker can compromise NS records to hijack the whole SLD when targeting recursive resolvers [54], [56]. In contrast, against forwarders, a malicious CNAME chain can be constructed to poison all domains along the chain [39], [40]. For

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

load balancing disruption, the SHAR attack can silence one or more authoritative nameservers, reducing their priority in resolver selection algorithms and consequently degrading or disrupting all domains hosted on the affected nameservers. Therefore, even when the triggered query domain contains special characters, it remains practicable to launch attacks against regular domain names.

5.4. Comparison with Prior Attacks

The SHAR attack is novel because it exploits two previously unknown logic flaws. These flaws were discovered through a systematic analysis of how special characters are handled across the DNS resolution chain. Previous work focused on specific parsing ambiguities, such as "\" and `0x00` characters [39], [40], to enable injection attacks. Other studies used side channels [60], [61] or shared resolver priorities [88] to poison caches or disrupt load balancing. However, no prior research has systematically examined how each DNS role processes the full ASCII set of special characters. Our work is the first to uncover the broader risks.

Moreover, the SHAR attack introduces new attack vectors and enhances 10/13 previous DNS cache poisoning techniques. In terms of technical methodology and impact, the two attacks (Four variants) operate differently from previous approaches as follows.

Cache poisoning attack. V_{EN} and V_{BF} exploit silent vulnerabilities introduced by handling special characters and subsequent response behavior. There is little discussion about the silent behavior. V_{EN} does not inject spoofed responses directly. Instead, it extends the attack window (over 60×) to enhance previous attacks by bypassing the race condition with legitimate replies (see Section 5.1.1). In contrast, V_{BF} differs from prior cache poisoning techniques that rely on advanced mechanisms such as side channels [56], [60], [61] or IP fragmentation [32], [81], [89]. Instead, V_{BF} leverages the newly discovered silent vulnerability to perform brute-force guessing of TxIDs and source ports, which has been considered infeasible since 2008. Furthermore, our end-to-end attack experiments confirm that V_{BF} reliably succeeds and can target all DNS roles and domains (TLD or SLD). Moreover, the time cost is entirely acceptable (see Section 5.1.2). We also observe poor randomization of source port or TxID in 14 software, which can further enhance V_{BF}.

Load balancing disruption attack. Both V_{LB} and V_{SF} exploit silent vulnerabilities to lower the priority of nameservers, thereby preventing them from being selected in subsequent queries. Unlike previous attacks [88] that rely on delegating unauthorized domains [2], [3], V_{LB} does not require the registration of new domains and can be executed directly via the victim domain. As a result, V_{LB} is more difficult to detect and mitigate. Moreover, V_{LB} can disrupt the entire TLD, posing a more severe real-world threat. When V_{LB} silences all nameservers, it can also trigger V_{SF}, resulting in a DoS effect. V_{SF} impacts all subsequent queries to the domain names hosted on these nameservers, causing the victim resolver to return invalid responses directly without even querying the nameservers.

6. Vulnerable DNS Components in the Wild

In this section, we conduct a comprehensive measurement of the SHAR attack in the wild, covering 21 Wi-Fi routers, 6 router OSes, 43 public DNS services, 883.5K open DNS resolvers, nameservers (Root, TLD, and SLD), and 223.6M domain names. For nameservers and domain names, we evaluate whether they are affected by the *silent* vulnerability. If affected, they are consequently vulnerable to all variants of SHAR attack. Our results show that the SHAR has a substantial and widespread real-world impact.

6.1. Wi-Fi Routers and Router OSes

As shown in Table 5, we selected 21 widely-used Wi-Fi routers along with 6 router operating systems (OSes) referenced in [78], [87] for evaluation.

Testing. Each router (or OS) was configured with our server as the upstream resolver. We tested whether they forward queries containing special characters and return a response when the upstream server remains silent. Since routers typically do not implement load balancing, our analysis focuses solely on the DNS cache poisoning attack. All testing experiments were conducted in real hardware environments under our control rather than in emulation.

Result. We categorize the vulnerability of V_{BF} into V_{BFs} and V_{BFd}. V_{BFs} refers to the router itself being affected, while V_{BFd} suggests that the router may expose the downstream resolvers to V_{BF}. As shown in Table 5, through systematic analysis, we found that V_{BFs} affects all routers. Furthermore, except RouterOS, all routers are also susceptible to V_{BFd}. In addition, we identify 11 router Wi-Fi routers or OSes with poor source port and TxID randomization, making them significantly more susceptible to V_{BF}.

6.2. Public DNS Services

Based on the measurements from APNIC [8], we selected 43 widely-used public DNS services and their corresponding IP addresses for evaluation, as shown in Table 6.

Testing. We tested these services for 5 rounds using domains under our control. For ethical considerations, our V_{BF} analysis focused solely on character handling without actual attacks. The query rate was limited to 200 queries per second. For load balancing disruption attacks, we sent 4K DNS queries on our controlled domain. Each domain was configured with 4 nameservers, consistent with the average number [56]. To test V_{LB} and V_{SF}, nameservers were configured to silently drop queries containing special characters. We then observed queries from all nameservers.

Result. As shown in Table 6, our results show that all tested public DNS services support queries containing special characters. Specifically, 11 services remained silent when no reply was received, which can expose downstream resolvers to V_{BF}. The results show that 26 services are susceptible to V_{BF}, including well-known services such as Cloudflare DNS, DNSPod Public DNS, and Quad9 DNS. Moreover, 25 services are vulnerable to V_{LB}, including AliDNS, Cisco

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

TABLE 5: Testing results of 21 Wi-Fi routers and 6 router OSes (All vulnerable).

WIFI Router and Router OS				Characters		Response Result			Vulnerability	
Role	Index	Software	Version	Unsupport	Support	Unsupport Response	Support Response	Status	V_{BFs}	V_{BFd}
Wi-Fi Router	# 1	360 WI-FI6 T7	4.2.4	0	65 (All)	–	✗	–	✓	✓
	# 2	ASUS RT-AC66U	384.18	0	65 (All)	–	✗	–	✓	✓
	# 3	CISCO Router	1.2.1.7	0	65 (All)	–	✗	–	✓*	✓
	# 4	D-Link 7001	17.01.11A1	1	64	✗	✗	–	✓	✓
	# 5	Fast FAC1200R	1.0.0	0	65 (All)	–	✗	–	✓	✓
	# 6	Fiberhome SR4201SA	RP0100	0	65 (All)	–	✗	–	✓*	✓
	# 7	H3C Magie NX15	100R012	0	65 (All)	–	✗	–	✓	✓
	# 8	Honor X4 Pro	16.0.0.38	0	65 (All)	–	✗	–	✓	✓
	# 9	Huawei AX3	4.0.0.19	33	32	✗	✗	–	✓	✓
	# 10	iKuai IK-Q3600	3.7.15	0	65 (All)	–	✗	–	✓*	✓
	# 11	Linksys	2.0.4.215745	1	64	✗	✗	–	✓	✓
	# 12	Mereury D191G	2.0.2	0	65 (All)	–	✗	–	✓	✓
	# 13	Netgenr AX5	1.0.8.82	0	65 (All)	–	✗	–	✓	✓
	# 14	PGY X4C-5131G	6.5.0	0	65 (All)	–	✗	–	✓	✓
	# 15	Redmi AX3000	1.0.68	0	65 (All)	–	✗	–	✓	✓
	# 16	Skyworth WR9651X	1.1.0	1	64	✓	✗	–	✓	✓
	# 17	Tenda V1	16.03.29.50	0	65 (All)	–	✗	–	✓	✓
	# 18	TP-Link XDR3230	1.0.22	0	65 (All)	–	✗	–	✓	✓
	# 19	TP-Link XDR5430	1.0.14	0	65 (All)	–	✗	–	✓†*	✓
	# 20	Xiaomi 4C	5.15.150	0	65 (All)	–	✗	–	✓	✓
	# 21	ZTE E2633	1.0.4	0	65 (All)	–	✗	–	✓†*	✓
Router OS	# 22	DD-WRT	v3.0-r39296	0	65 (All)	–	✗	–	✓*	✓
	# 23	Gargoyle	1.14.0	0	65 (All)	–	✗	–	✓*	✓
	# 24	iKuai OS	3.7.17	0	65 (All)	–	✗	–	✓*	✓
	# 25	libreCMC	1.5.15	0	65 (All)	–	✗	–	✓*	✓
	# 26	Openwrt	23.05.3	0	65 (All)	–	✗	–	✓*	✓
	# 27	RouterOS	7.16.2	0	65 (All)	–	✓	Servfail	✓*	✗

"–": Applicable due to no unsupported characters or no response. ✓: Yes. ✗: No. ✓: Vulnerable. ✗: Not vulnerable.
"*": Poor randomization of source port. "†": Poor randomization of TxID.

OpenDNS, and Quad9 DNS. In addition, 16 services are affected by V_{SF}, such as AdGuard DNS, DNS for Family, and Level3 DNS. We observe that 13 services support *qmin*. However, although *qmin* constrains SHAR at the TLD level, it remains effective at the SLD level. Further discussion is provided in Section 5.3. These findings show that all tested public DNS services are affected by SHAR, with 25 services vulnerable to at least two attack variants.

6.3. Open DNS Resolvers

Since open DNS resolvers tend to be volatile [80], our goal is to identify stable resolvers that remained consistently active (from April 5, 2025, to May 5, 2025). To this end, we scanned the IPv4 address space on UDP port 53 using XMap [53], targeting our controlled domain. Resolvers returning incorrect responses were excluded. In total, we identified 883.5K resolvers, including both forwarders and recursive resolvers. These resolvers are distributed across 228 regions and 18,142 autonomous systems (ASes), as identified by the MaxMind GeoLite2 database [63]. China and the United States had the highest number of resolvers

Testing. To analyze the vulnerability of all these DNS resolvers to SHAR, we refrained from conducting actual attacks due to ethical considerations. Instead, we performed the following steps: *(i)* We send DNS queries containing special characters to these resolvers using our controlled domains. *(ii)* By observing the queries from controlled nameservers, we can identify which characters are supported by each resolver. *(iii)* Additionally, we analyze whether the resolver remains silent when all nameservers directly drop the queries. Given that all known implementations are exploitable even in their latest versions, identifying *(ii)* and *(iii)* is sufficient to determine the potential vulnerability.

Result. We identified 531.1K (60.1%) resolvers vulnerable to the SHAR attack. Among them, 115.6K (13.1%) resolvers satisfy *(ii)*, indicating that they are vulnerable. 515.3K (58.3%) resolvers satisfy *(iii)*, indicating they make downstream resolvers vulnerable. Only 0.1% of open DNS resolvers support *0x20*, and only 0.03% support *qmin*.

6.4. Nameservers (Root, TLD, and SLD)

Nameservers typically include Root, TLD, and SLD servers. We systematically analysed all 13 IPv4 and IPv6 Root nameservers [35], all 1,445 TLD (March 2025, 14,660 nameservers) [37], and the Tranco Top-1M SLD's nameservers (collected throughout March 2025) [73].

Testing. Instead of launching actual attacks, we focused on identifying which special characters can cause the nameserver to drop DNS queries. If the nameserver can respond to a normal query but fails to respond to a query containing special characters within 5s, we consider the nameserver to be vulnerable. In particular, to avoid the effects of anycast and load-based routing, we conducted measurements from three vantage points, including Oregon (US), Frankfurt (Germany), and Singapore, for 5 rounds.

Result. *Root nameservers.* None of the root servers were affected by these vulnerabilities, further demonstrating their high management standards [35], [44].

Top-level domain nameservers. We obtained the nameservers corresponding to each TLD from the root server [35],

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

TABLE 6: Testing results of 43 public DNS services (All vulnerable).

Public DNS Service			Characters		Response Result[3]			Mechanism		Vulnerability[8]		
Index	Vendor	IP	Unsup.[1]	Sup.[2]	Unsup. Resp.[4]	Support Resp.[5]	Support Status[6]	0x20	Qmin.[7]	V_{BF}	V_{LB}†	V_{SF}*
# 1	114DNS	114.114.114.114	34	31	✗	✗	–	✗	✗	✓	✗	✗
# 2	360 Secure DNS	101.226.4.6	2	63	✓	✓	Servfail	✗	✗	✗	✗	✓
# 3	AdGuard DNS	94.140.14.14	0	65 (All)	–	✓	Servfail	✗	✗	✓	✓	✗
# 4	Akamai Vantio DNS	23.56.160.142	0	65 (All)	–	✓	Servfail	✗	✗	✓	✗	✗
# 5	Ali DNS	223.5.5.5	0	65 (All)	–	✓	Servfail	✗	✗	✗	✓	✓
# 6	Baidu DNS	180.76.76.76	0	65 (All)	–	✓	Servfail	✗	✗	✓	✗	✗
# 7	ByteDance DNS	180.184.1.1	63	2	✓	✗	–	✗	✗	✓	✗	✗
# 8	CenturyLink DNS	205.171.3.66	0	65 (All)	–	✗	–	✗	✗	✓	✓	✗
# 9	CIRA Shield DNS	149.112.121.10	0	65 (All)	–	✗	–	✗	✗	✓	✓	✗
# 10	Cisco OpenDNS	208.67.222.222	0	65 (All)	–	✓	Servfail	✗	✓	✓	✓	✗
# 11	CleanBrowsing	185.228.168.10	0	65 (All)	–	✓	Servfail	✗	✗	✗	✓	✓
# 12	CloudFlare DNS	1.1.1.1	0	65 (All)	–	✓	Servfail	✗	✗	✓	✗	✗
# 13	CNNIC sDNS	1.2.4.8	0	65 (All)	–	✗	–	✗	✗	✓	✗	✗
# 14	Comodo Secure	8.26.56.10	0	65 (All)	–	✓	Servfail	✗	✗	✗	✓	✓
# 15	Comss.one DNS	195.133.25.16	0	65 (All)	–	✓	Servfail	✗	✓	✗	✗	✓
# 16	ControlD DNS	76.76.2.5	62	3	✓	✓	Formerr	✗	✗	✗	✓	✓
# 17	CZ.NIC ODVR	193.17.47.1	0	65 (All)	–	✓	Servfail	✓	✓	✗	✓	✓
# 18	DNS for Family	94.130.180.225	0	65 (All)	–	✓	Servfail	✓	✗	✗	✗	✓
# 19	DNS Forge	176.9.93.198	0	65 (All)	–	✓	–	✗	✓	✓	✓	✗
# 20	DNS.SB	185.222.222.222	0	65 (All)	–	✓	Servfail	✗	✗	✓	✓	✗
# 21	DNS.WATCH	84.200.69.80	0	65 (All)	–	✗	–	✗	✗	✓	✗	✗
# 22	DNSPod DNS+	119.28.28.28	62	3	✓	✓	NXDomain	✗	✗	✓	✗	✗
# 23	Dyn DNS	216.146.35.35	0	65 (All)	–	✓	Servfail	✗	✗	✗	✗	✓
# 24	G-Core DNS	95.85.95.85	0	65 (All)	–	✓	Servfail	✗	✗	✓	✓	✗
# 25	Google DNS	8.8.8.8	0	65 (All)	–	✓	Servfail	✓	✗	✗	✗	✗
# 26	Hurricane DNS	74.82.42.42	0	65 (All)	–	✓	Servfail	✗	✓	✗	✗	✓
# 27	Level3 DNS	4.2.2.1	0	65 (All)	–	✓	Servfail	✓	✓	✗	✗	✓
# 28	LibreDNS	88.198.92.222	0	65 (All)	–	✓	Servfail	✗	✓	✓	✓	✗
# 29	Neustar UltraDNS	156.154.70.1	0	65 (All)	–	✓	Servfail	✗	✓	✗	✓	✓
# 30	NextDNS	45.90.30.118	0	65 (All)	–	✓	Servfail	✗	✗	✓	✓	✗
# 31	Norton ConnectSafe	199.85.126.10	0	65 (All)	–	✓	Servfail	✗	✗	✗	✓	✓
# 32	OneDNS	52.80.66.66	0	65 (All)	–	✗	–	✗	✗	✓	✓	✗
# 33	OpenNIC DNS	103.1.206.179	0	65 (All)	–	✗	–	✗	✓	✓	✗	✗
# 34	Quad101 DNS	101.101.101.101	0	65 (All)	–	✗	–	✗	✓	✓	✓	✗
# 35	Quad9 DNS	9.9.9.11	0	65 (All)	–	✓	Servfail	✗	✓	✓	✓	✗
# 36	SafeDNS	195.46.39.39	0	65 (All)	–	✓	Servfail	✗	✗	✗	✓	✓
# 37	SafeSurfer DNS	104.197.28.121	0	65 (All)	–	✓	Servfail	✗	✓	✓	✓	✗
# 38	SkyDNS	193.58.251.251	0	65 (All)	–	✓	Servfail	✗	✗	✗	✓	✓
# 39	Strongarm DNS	52.3.100.184	0	65 (All)	–	✓	Servfail	✗	✗	✓	✓	✗
# 40	Surfshark	194.169.169.169	0	65 (All)	–	✓	Servfail	✗	✗	✓	✗	✗
# 41	Tiarap Public DNS	174.138.21.128	0	65 (All)	–	✓	NXDomain	✓	✓	✗	✓	✓
# 42	Verisign Public DNS	64.6.65.6	0	65 (All)	–	✓	Servfail	✗	✗	✓	✓	✗
# 43	Yandex DNS	77.88.8.1	0	65 (All)	–	✗	–	✗	✗	✓	✗	✗

[1]: # of unsupported characters. [2]: # of supported special characters. [3]: Response behavior to the query. No response will expose downstream components to the vulnerabilities. [4]: Whether unsupported characters can trigger responses. "-": Not applicable because the service supports all characters. [5]: Whether a response is made within 5s. [6]: RCODE in the response when the upstream nameservers keep silent. [7]: Query name minimization. ✓: Supported. ✗: Unsupported. [8]: Whether the service is affected by the attack. ✓: Vulnerable. ✗: Not vulnerable. "†": If 90%+ queries are disrupted to a single nameserver, mark ✓. (See Table 9 in Appendix B). "*": If all subsequent queries directly return *ServFail* without sending requests to the nameservers, mark ✓.

TABLE 7: Experiment results of all TLD nameservers.

Variety [36]	Num	>10%†	>25%†	>50%†	>75%†	100%†
ccTLD	236	95	58	17	5	4
IDN TLD	151	75	52	23	3	2
gTLD	1,058	727	598	482	1	1
Total (Num)	1,445	897	708	522	9	7
Total (%)	**100.0%**	**62.1%**	**49.0%**	**36.1%**	**0.6%**	**0.5%**

"†": Fraction of vulnerable nameservers per TLD. More vulnerable NS implies a more severe impact. TLDs with over 50% vulnerable NS are susceptible to V_{LB}, V_{EN} and V_{BF}. 100% are susceptible to V_{SF}.

and tested them for 5 rounds. If a nameserver drops queries containing specific characters, we consider the corresponding TLD vulnerable, as the attacker can concentrate all queries on the remaining nameservers within that TLD. As shown in Table 7, we quantify the proportion of vulnerable nameservers within each TLD. The results show that V_{LB} can affect 522 (36.1%) TLDs, including `.info`, `.ai`, and `.io` and V_{SF} affect 7 (0.5%) TLDs, including `.ir` and `.mil`.

Second-level domain nameservers. We recursively traversed the resolution topology of all domains in the Tranco Top-1M lists to extract SLD nameservers. We identified 108.5K servers that are consistently active (5 April 2025 to 7 May 2025), and 5.6K (5.2%) are vulnerable. The attacker can cause a DoS effect on these servers by SHAR attack. The vulnerable cloud providers include Alibaba Cloud, Microsoft Azure, Tencent Cloud, and DNS.COM.

6.5. Domain Names

To further evaluate the real-world impact of the SHAR attack, we used two methods, `TLDScope` and `SLD-NScope`. Under `TLDScope`, all domains under the affected TLD are considered vulnerable, because domain resolution depends on the availability of that TLD's servers. If more than 50% of the domain's SLD nameservers are affected, `SLD-NScope` marks the domain vulnerable. We obtained 1,093 TLD zone files from the Centralized Zone Data Service (CZDS) [38],

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

and extracted a total of 223.6M domain names. As shown in Table 8, we identified 12.5M (5.6%) domain names that are vulnerable to the SHAR attack, affecting nearly all sectors [67], including education, healthcare, and finance.

TABLE 8: Experiment results of domain names.

Total Num	TLDScope†	SLD-NScope*	Total Vulnerable
223.6M	11.4M (5.1%)	1.2M (0.5%)	12.5M (5.6%)

"†": Identified by TLD with over 50% NS vulnerable.
"*": Identified by SLD with over 50% NS vulnerable.

7. Discussion and Mitigation

In this section, we first describe how we address ethical considerations, then analyze the lessons learned from the *silent* vulnerabilities, propose detailed mitigation solutions to defend against the shar attack, and lastly report our responsible disclosure results.

7.1. Ethical considerations

Since our experiments involved active resolvers and potentially serious vulnerabilities, we strictly adhered to ethical guidelines outlined in the Menlo Report [45] and community best practices for network measurement studies [70]. *(i)* All DNS software and OSes were installed and tested on our machines. *(ii)* Regarding 43 public DNS services and open DNS resolvers, we only probed their IPs and strictly controlled the probing speed. All queries were sent from our domain names and observed on our controlled nameservers. Additionally, we configured the `PTR` records and hosted a webpage at the probing IP to clearly show our research objectives, and no opt-out requests were received. Finally, we disclosed all vulnerabilities to the affected vendors.

7.2. Lessons learned

DNS characters are the fundamental building blocks of domain names, and correct handling of special characters is critical to DNS security. We discovered novel *silent* vulnerabilities exploitable by attackers to launch cache poisoning and load balancing disruption attacks, posing significant real-world threats. In particular, if a nameserver silently drops queries containing special characters, downstream components such as resolvers, forwarders, and stub resolvers may remain waiting. This behavior creates a time window that can be exploited. We analyzed the default configurations of widely-used nameserver software and found that they typically respond to such queries (see Section 4.2). However, in the real world, approximately 36.1% TLD and 5.2% SLD servers silently drop these queries, including major cloud providers such as Alibaba Cloud, Microsoft Azure, and Tencent Cloud (see Section 6.4). They adopt stricter handling logic by dropping queries containing unsupported characters, inadvertently exposing an attack surface to attackers. Our work presents a comprehensive analysis of how different DNS software handles special characters and demonstrates that the SHAR attack has a significant real-world impact. We leave further exploration of ambiguities in handling all special characters for future work.

7.3. Mitigation solutions

SHAR exploits vulnerabilities in DNS character handling. The root cause lies in discrepancies between RFC 1034 [2] and RFC 2181 [23], where RFC 1034 restricts domain names to (`a-z, A-Z, 0-9`) and hyphens (-), whereas RFC 2181 permits all characters. To mitigate these novel vulnerabilities, we recommend that DNS roles fully support all characters as per the latest RFC specifications [23]. Furthermore, for unsupported characters or when upstream nameservers do not respond, the resolver should reply with a response (`Servfail` or `Refused`).

To defend against cache poisoning, we advocate enabling 0x20 encoding [21] and deploying DNSSEC [77] to cryptographically sign DNS responses. Additionally, resolvers can enhance security by blocking malicious response injections through anomaly detection and rate-limiting techniques, as implemented by PowerDNS Recursor [75]. To mitigate load balancing disruptions, we recommend that resolvers periodically poll lower-priority nameservers.

7.4. Responsible Disclosure

We disclosed the *silent* vulnerability details to all affected vendors. To date, we have received responses from 18 vendors. All of them acknowledged the issue and engaged in discussions with us regarding mitigation strategies. For those who have not yet responded, we are actively communicating with them. The progress updates are as follows:

- **BIND**: Received our disclosure and stated that "*it is actually a security vulnerability in these products as they increase the window that an attacker can use for spoofing the answers*". However, they noted that the most expedient short-term mitigation would be for authoritative nameservers to support all characters rather than discarding such queries outright.
- **Unbound**: Acknowledged the SHAR attack and characterized it as a new attack paradigm. They emphasized that "wrong characters" are not strictly against the DNS protocol and that DNS software should support such characters. In addition, they argued that treating a nameserver as unavailable simply because it does not respond to certain queries is not a reasonable approach.
- **PowerDNS**: Acknowledged the vulnerability but attributed the root cause to nameservers failing to respond. They have already adjusted their algorithm so that such nameservers will no longer be fully ignored.
- **Microsoft DNS**: Confirmed the vulnerability, assigned a VULN ID, and ongoing discussions are in progress.
- **JavaDNS, YogaDNS, dnsmasq**: Acknowledged the vulnerability, but emphasized that upstream resolvers should not silently discard such queries.
- **Technitium DNS, AdGuard/AdGuard Home, TP-Link, Quad9, 114 DNS**: Received our disclosure and expressed interest. They are sending staff to conduct testing and remediation.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

8. Conclusion

In this paper, we systematically analyze the handling of special characters by reviewing relevant RFCs and 31 DNS implementations through source code review and gray-box testing, uncovering two novel logic vulnerabilities. Based on these insights, we propose the SHAR attack, which targets all DNS roles and enhances the previous DNS cache poisoning attack. The results show that SHAR can affect all 31 tested software. We further conducted large-scale measurements to identify the real-world impact. The results show that 43/43 public DNS services, 21/21 WIFI routers, 6/6 route OSes, 531.1K (60.1%) resolvers, 522 (36.1%) TLD, and over 12.5M (5.6%) domain names are vulnerable to the SHAR attack.

Acknowledgement

We are grateful to the anonymous reviewers, our shepherd, and the Research Ethics Committee for their insightful feedback, and we especially appreciate our shepherd's patient and thoughtful guidance, which greatly helped us enhance this paper. Authors (Prof. Xiang Li) from Nankai University were supported by the National Natural Science Foundation of China (No. 6250072745) and the Natural Science Foundation of Tianjin (No. 24JCQNJC02070). Authors from Tsinghua University were supported by the National Key Research and Development Program of China under Grant 2020YFE0200500.

References

[1] DoD Internet host table specification. RFC 952, 1985.

[2] Domain names - concepts and facilities. RFC 1034, 1987.

[3] Domain names - implementation and specification. RFC 1035, 1987.

[4] Johnny Al Helou and Scott Tilley. Multilingual web sites: Internationalized domain name homograph attacks. In *WSE '10*, 2010.

[5] Mark Allman. Comments on dns robustness. In *IMC '18*, 2018.

[6] Sumayah Alrwais, Xiaojing Liao, Xianghang Mi, Peng Wang, XiaoFeng Wang, Feng Qian, Raheem Beyah, and Damon McCoy. Under the shadow of sunshine: Understanding and detecting bulletproof hosting on legitimate service provider networks. In *S&P '17*, 2017.

[7] Mark P. Andrews. Negative Caching of DNS Queries (DNS NCACHE). RFC 2308, 1998.

[8] APNIC. DNS Resolvers Use. https://stats.labs.apnic.net/rvrs, 2025.

[9] BIND. https://github.com/isc-projects/bind9/blob/main/doc/arm/reference.rst, 2025.

[10] Bind. dns_name_fromwire. https://github.com/isc-projects/bind9/blob/main/lib/dns/name.c#L1286, 2025.

[11] Stéphane Bortzmeyer. DNS Query Name Minimisation to Improve Privacy. RFC 7816, 2016.

[12] Robert T. Braden. Requirements for Internet Hosts - Application and Support. RFC 1123, 1989.

[13] Thomas P. Brisco. DNS Support for Load Balancing. RFC 1794, 1995.

[14] Jonas Bushart and Christian Rossow. DNS Unchained: Amplified Application-Layer DoS Attacks Against DNS Authoritatives. In *RAID '18*, 2018.

[15] CAIDA. State of IP Spoofing. https://spoofer.caida.org/summary.php, 2025.

[16] ChromeOS. isValid. https://github.com/GoogleChrome/lighthouse/blob/c84cd54a/core/audits/valid-source-maps.js#L60, 2025.

[17] Cloudflare. What is dns cache poisoning? — dns spoofing. https://www.cloudflare.com/learning/dns/dns-cache-poisoning/, 2025.

[18] CNET. AlterNIC Takes over InterNIC Traffic. https://www.cnet.com/tech/mobile/alternic-takes-over-internic-traffic/, 1997.

[19] Adam M. Costello. Punycode: A Bootstring encoding of Unicode for Internationalized Domain Names in Applications (IDNA). RFC 3492, 2003.

[20] David Dagon, Manos Antonakakis, Kevin Day, Xiapu Luo, Christopher P Lee, and Wenke Lee. Recursive dns architectures and vulnerability implications. In *NDSS '09*, 2009.

[21] David Dagon, Manos Antonakakis, Paul Vixie, Tatuya Jinmei, and Wenke Lee. Increased dns forgery resistance through 0x20-bit encoding: security via leet queries. In *CCS '08*, 2008.

[22] Dnsmasq. extract_name. https://github.com/imp/dnsmasq/blob/master/src/rfc1035.c#L19, 2025.

[23] Robert Elz and Randy Bush. Clarifications to the DNS Specification. RFC 2181, 1997.

[24] Patrik Fältström. The Unicode Code Points and Internationalized Domain Names for Applications (IDNA). RFC 5892, 2010.

[25] Patrik Fältström and Paul E. Hoffman. Internationalizing Domain Names in Applications (IDNA). RFC 3490, 2003.

[26] Evgeniy Gabrilovich and Alex Gontmakher. The homograph attack. *Communications of the ACM*, 2002.

[27] GoDaddy. GoDaddy. https://www.godaddy.com/, 2025.

[28] Golang. miekg. https://github.com/miekg/dns, 2025.

[29] Shuai Hao, Yubao Zhang, Haining Wang, and Angelos Stavrou. {End-Users} get maneuvered: Empirical analysis of redirection hijacking in content delivery networks. In *USENIX Security '18*, 2018.

[30] Elias Heftrig, Haya Shulman, and Michael Waidner. Downgrading {DNSSEC}: How to exploit crypto agility for hijacking signed zones. In *USENIX Security '23*, 2023.

[31] Amir Herzberg and Haya Shulman. Security of patched dns. In *ESORICS'12*, 2012.

[32] Amir Herzberg and Haya Shulman. Fragmentation considered poisonous, or: One-domain-to-rule-them-all.org. In *CNS '13*, 2013.

[33] Amir Herzberg and Haya Shulman. Socket overloading for fun and cache-poisoning. In *Proceedings of the 29th Annual Computer Security Applications Conference*, 2013.

[34] Amir Herzberg and Haya Shulman. Vulnerable delegation of dns resolution. In *Computer Security–ESORICS 2013*, 2013.

[35] Internet Assigned Numbers Authority. Root servers. https://www.iana.org/domains/root/servers, 2025.

[36] Internet Assigned Numbers Authority. Root zone database. https://www.iana.org/domains/root/db, 2025.

[37] Internet Assigned Numbers Authority. Root zone database - list of top-level domains. https://data.iana.org/TLD/tlds-alpha-by-domain.txt, 2025.

[38] Internet Corporation for Assigned Names and Numbers (ICANN). Icann centralized zone data service (czds). https://czds.icann.org/zone-requests/all, 2025.

[39] Philipp Jeitner and Haya Shulman. Injection attacks reloaded: Tunnelling malicious payloads over DNS. In *USENIX Security '21*, 2021.

[40] Philipp Jeitner, Haya Shulman, Lucas Teichmann, and Michael Waidner. XDRI attacks - and - how to enhance resilience of residential routers. In *USENIX Security '22*, Boston, MA, 2022.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

[41] Dan Kaminsky. Black ops 2008: It's the end of the cache as we know it. *Black Hat USA*, 2008.

[42] Dan Kaminsky. It's the end of the cache as we know it. https://www.blackhat.com/presentations/bh-jp-08/bh-jp-08-Kaminsky/BlackHatJapan-08-Kaminsky-DNS08-BlackOps.pdf, 2008. Black Hat Japan.

[43] Cary Karp and Harald T. Alvestrand. Right-to-Left Scripts for Internationalized Domain Names for Applications (IDNA). RFC 5893, 2010.

[44] Daniel Karrenberg, Mark Kosters, Raymond Plzak, and Randy Bush. Root Name Server Operational Requirements. RFC 2870, 2000.

[45] Erin Kenneally and David Dittrich. The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research. *SSRN Electronic Journal*, 2012.

[46] Amit Klein. Windows dns server cache poisoning. Technical report, Trusteer, 2007.

[47] Amit Klein. Powerdns recursor dns cache poisoning. Technical report, Trusteer, 2008.

[48] Dr. John C. Klensin. Internationalized Domain Names for Applications (IDNA): Background, Explanation, and Rationale. RFC 5894, 2010.

[49] Dr. John C. Klensin. Internationalized Domain Names for Applications (IDNA): Definitions and Document Framework. RFC 5890, 2010.

[50] Dr. John C. Klensin. Internationalized Domain Names in Applications (IDNA): Protocol. RFC 5891, 2010.

[51] Knot Resolver. knot_dname_from_str. https://gitlab.nic.cz/knot/knot-dns/-/blob/master/src/libknot/dname.c#L324, 2025.

[52] Xiang Li, Baojun Liu, Xuesong Bai, Mingming Zhang, Qifan Zhang, Zhou Li, Haixin Duan, and Qi Li. Ghost domain reloaded: Vulnerable links in domain name delegation and revocation. In *NDSS '23*, 2023.

[53] Xiang Li, Baojun Liu, Xiaofeng Zheng, Haixin Duan, Qi Li, and Youjun Huang. Fast ipv6 network periphery discovery and security implications. In *DSN '21*, 2021.

[54] Xiang Li, Chaoyi Lu, Baojun Liu, Qifan Zhang, Zhou Li, Haixin Duan, and Qi Li. The maginot line: Attacking the boundary of {DNS} caching protection. In *USENIX Security '23*, 2023.

[55] Xiang Li, Dashuai Wu, Haixin Duan, and Qi Li. Dnsbomb: A new practical-and-powerful pulsing dos attack exploiting dns queries-and-responses. In *S&P '24*, 2024.

[56] Xiang Li, Wei Xu, Baojun Liu, Mingming Zhang, Zhou Li, Jia Zhang, Deliang Chang, Xiaofeng Zheng, Chuhan Wang, Jianjun Chen, et al. Tudoor attack: Systematically exploring and exploiting logic vulnerabilities in dns response pre-processing with malformed packets. In *S&P '24*, 2024.

[57] LinuxLinks. Best free and open source dns servers. https://www.linuxlinks.com/best-free-open-source-dns-servers/, 2023.

[58] Baojun Liu, Chaoyi Lu, Haixin Duan, Ying Liu, Zhou Li, Shuang Hao, and Min Yang. Who is answering my queries: Understanding and characterizing interception of the {DNS} resolution path. In *USENIX Security '18*, 2018.

[59] Baojun Liu, Chaoyi Lu, Zhou Li, Ying Liu, Haixin Duan, Shuang Hao, and Zaifeng Zhang. A reexamination of internationalized domain names: The good, the bad and the ugly. In *DSN '18*, 2018.

[60] Keyu Man, Zhiyun Qian, Zhongjie Wang, Xiaofeng Zheng, Youjun Huang, and Haixin Duan. Dns cache poisoning attack reloaded: Revolutions with side channels. In *CCS '20*, 2020.

[61] Keyu Man, Xin'an Zhou, and Zhiyun Qian. Dns cache poisoning attack: Resurrections with side channels. In *CCS '21*, 2021.

[62] Maradns. decomp_get_label. https://github.com/samboy/MaraDNS/blob/master/dns/Decompress.c#L68, 2025.

[63] MaxMind. GeoLite2 Free Geolocation Data. https://dev.maxmind.com/geoip/geolite2-free-geolocation-data, 2023.

[64] Xianghang Mi, Xuan Feng, Xiaojing Liao, Baojun Liu, XiaoFeng Wang, Feng Qian, Zhou Li, Sumayah Alrwais, Limin Sun, and Ying Liu. Resident evil: Understanding residential ip proxy as a dark service. In *S&P '19*, 2019.

[65] Microsoft DNS. https://learn.microsoft.com/en-us/windows-server/networking/dns/dns-overview, 2025.

[66] Giovane CM Moura, Sebastian Castro, John Heidemann, and Wes Hardaker. TsuNAME: Exploiting Misconfiguration and Vulnerability to DDoS DNS. In *IMC '21*, 2021.

[67] MSCI Inc. Global industry classification standard (gics). https://www.msci.com/indexes/index-resources/gics, 2025.

[68] Moritz Müller, Giovane CM Moura, Ricardo de O. Schmidt, and John Heidemann. Recursives in the wild: Engineering authoritative dns servers. In *IMC 17*, 2017.

[69] Vasileios Pappas, Zhiguo Xu, Songwu Lu, Daniel Massey, Andreas Terzis, and Lixia Zhang. Impact of configuration errors on dns robustness. In *SIGCOMM '04*, 2004.

[70] Craig Partridge and Mark Allman. Ethical Considerations in Network Measurement Papers. *Communications of the ACM*, 2016.

[71] Olga Pearce, Todd Gamblin, Bronis R De Supinski, Martin Schulz, and Nancy M Amato. Quantifying the effectiveness of load balance algorithms. In *ICS' 12*, 2012.

[72] Pi-hole. extract_name. https://github.com/pi-hole/dnsmasq/blob/master/src/rfc1035.c#L35, 2025.

[73] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. Tranco: A research-oriented top sites ranking hardened against manipulation. *arXiv preprint arXiv:1806.01156*, 2018.

[74] PowerDNS. packetParser. https://github.com/PowerDNS/pdns/blob/master/pdns/dnsname.cc#L182, 2025.

[75] PowerDNS. *spoof-nearmiss-max*, 2025. https://docs.powerdns.com/recursor/settings.html#spoof-nearmiss-max.

[76] RIPE. RIPE Atlas. https://atlas.ripe.net/, 2025.

[77] Scott Rose, Matt Larson, Dan Massey, Rob Austein, and Roy Arends. DNS Security Introduction and Requirements. RFC 4033, 2005.

[78] RouterChart. Popular Routers. https://routerchart.com/brands, 2025.

[79] Vagner Sacramento. Vulnerability in requests control of bind versions 4 and 8 allows dns spoofing. https://lists.isc.org/pipermail/bind-users/2002-November/043141.html, 2002.

[80] Kyle Schomp, Tom Callahan, Michael Rabinovich, and Mark Allman. On Measuring the Client-side DNS Infrastructure. In *IMC '13*, 2013.

[81] Haya Shulman and Michael Waidner. Fragmentation considered leaking: port inference for dns poisoning. In *ACNS '14*, 2014.

[82] Sooel Son and Vitaly Shmatikov. The hitchhiker's guide to dns cache poisoning. In *SecureComm' 10*, 2010.

[83] Linux Today Staff. 21 best free and open source dns servers, 2025.

[84] Ao-Jan Su and Aleksandar Kuzmanovic. Thinning akamai. In *IMC '08*, 2008.

[85] Technitium. packetParser. https://github.com/TechnitiumSoftware/DnsServer/blob/master/DnsServerCore/Dns/Trees/DomainTree.cs#L108, 2025.

[86] Unbound. dname_valid. https://github.com/NLnetLabs/unbound/blob/master/util/data/dname.c#L74, 2025.

[87] Wikipedia contributors. List of Router Firmware Projects. https://en.wikipedia.org/wiki/List_of_router_firmware_projects, 2025.

[88] Fenglu Zhang, Baojun Liu, Eihal Alowaisheq, Jianjun Chen, Chaoyi Lu, Linjian Song, Yong Ma, Ying Liu, Haixin Duan, and Min Yang. Silence is not golden: Disrupting the load balancing of authoritative dns servers. In *CCS '23*, 2023.

[89] Xiaofeng Zheng, Chaoyi Lu, Jian Peng, Qiushi Yang, Dongjie Zhou, Baojun Liu, Keyu Man, Shuang Hao, Haixin Duan, and Zhiyun Qian. Poison over troubled forwarders: A cache poisoning attack targeting {DNS} forwarding devices. In *USENIX Security '20*, 2020.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

Appendix A.
Domain Name Validation Algorithm

The overall domain name validation algorithm is shown in Algorithm 1.

Algorithm 1: Domain Name Validation

input : Raw DNS `Packet`, Validation `Options`
output: Validation Result

```
ValidateDNSMessage(Packet)
    Result ← WireFormatParsing(Packet)
    if Result ≠ SUCCESS then
        return PARSE_ERROR
    Result ← BasicFormatValidation(Packet)
    if Result ≠ SUCCESS then
        return FORMAT_ERROR
    foreach Name ∈ Packet.names do
        Result ← SemanticValidation(Name)
        if Result ≠ SUCCESS then
            return SEMANTIC_ERROR
    Result ← PolicyValidation(Packet, Options)
    if Result ≠ SUCCESS then
        return POLICY_ERROR
    return SUCCESS
WireFormatParsing(Packet)
    Parse and decompress names
    Validate label length, pointer, total name length
    return Status
BasicFormatValidation(Packet)
    Check header and section integrity
    Ensure names terminate with root label
    return Status
SemanticValidation(Name)
    Validate characters per RFC rules
    Check special name types if applicable
    return Status
PolicyValidation(Packet, Options)
    foreach RR ∈ Packet.resource_records do
        Validate RR.owner_name per policy
        Take action based on policy (FAIL, WARN, IGNORE)
    return Status
```

Appendix B.
Eperiment Results Of V_{LB}

The experimental results of the V_{LB} attack are shown in Table 9. We deployed 4 nameservers (S_1, S_2, S_3 and victim), all under our control. S_1, S_2, and S_3 were configured to silently drop queries containing special characters, while the victim was configured to support all characters and respond normally. We continuously sent 4,000 queries and recorded the number of queries received by each nameserver. If more than 90% of queries are concentrated on the victim, we consider the tested service vulnerable to the V_{LB} attack. Moreover, we observed that 114DNS, Baidu DNS, and CNNIC DNS exhibit a strong preference for selecting the first two NS RRs in DNS responses. Consequently, the victim received even fewer than the expected average (25%).

TABLE 9: Experiment results of public DNS services (V_{LB}, 25 Vulnerable).

Vendor	IP	S_1	S_2	S_3	Victim
114DNS	114.114.114.114	0.3%	48.1%	51.2%	0.4%
360 Secure DNS	101.226.4.6	5.6%	3.1%	2.8%	88.5%
AdGuard DNS	94.140.14.14	0.0%	0.0%	0.0%	100.0%
Akamai Vantio DNS	23.56.160.142	20.8%	20.6%	20.7%	37.9%
Ali DNS	223.5.5.5	0.9%	0.2%	0.2%	98.6%
Baidu DNS	180.76.76.76	1.4%	45.6%	44.5%	8.5%
ByteDance DNS	180.184.1.1	11.3%	10.4%	11.1%	67.2%
CenturyLink DNS	205.171.3.66	0.2%	0.1%	0.1%	99.6%
CIRA Shield DNS	149.112.121.10	0.6%	0.7%	0.9%	97.9%
Cisco OpenDNS	208.67.222.222	1.1%	1.1%	1.0%	96.8%
CleanBrowsing	185.228.168.10	0.4%	0.6%	0.8%	98.2%
CloudFlare DNS	1.1.1.1	10.1%	6.4%	8.9%	74.6%
CNNIC sDNS	1.2.4.8	0.1%	49.8%	49.6%	0.5%
Comodo Secure	8.26.56.10	0.1%	0.1%	0.1%	99.7%
Comss.one DNS	195.133.25.16	4.2%	4.2%	4.2%	87.3%
ControlD DNS	76.76.2.5	1.4%	2.0%	1.2%	95.4%
CZ.NIC ODVR	193.17.47.1	2.2%	2.2%	2.2%	93.5%
DNS for Family	94.130.180.225	16.9%	16.5%	16.4%	50.1%
DNS Forge	176.9.93.198	1.6%	1.5%	1.1%	95.8%
DNS.SB	185.222.222.222	0.9%	1.3%	1.2%	96.7%
DNS.WATCH	84.200.69.80	26.7%	29.6%	27.4%	16.4%
DNSPod Public DNS	119.28.28.28	0.5%	0.5%	0.5%	98.5%
Dyn	216.146.35.35	9.9%	10.2%	9.6%	70.4%
G-Core DNS	95.85.95.85	1.0%	1.0%	1.0%	97.0%
Google DNS	8.8.8.8	16.3%	16.3%	16.1%	51.3%
Hurricane DNS	74.82.42.42	7.6%	8.0%	7.6%	76.9%
Level3 DNS	4.2.2.1	13.7%	14.0%	14.4%	57.8%
LibreDNS	88.198.92.222	1.1%	0.6%	0.7%	97.7%
Neustar UltraDNS	156.154.70.1	0.0%	0.1%	0.0%	99.9%
NextDNS	45.90.30.118	0.0%	0.0%	0.0%	100.0%
Norton ConnectSafe	199.85.126.10	0.0%	0.0%	0.1%	99.9%
OneDNS	52.80.66.66	0.6%	0.3%	0.3%	98.8%
OpenNIC	103.1.206.179	14.8%	15.7%	16.3%	53.2%
Quad101 DNS	101.101.101.101	0.9%	2.3%	2.2%	94.6%
Quad9 DNS	9.9.9.11	2.1%	2.7%	2.7%	92.5%
SafeDNS	195.46.39.39	0.1%	0.1%	0.2%	99.5%
SafeSurfer DNS	104.197.28.121	0.3%	0.5%	0.4%	98.9%
SkyDNS	193.58.251.251	0.2%	0.2%	0.2%	99.3%
Strongarm DNS	52.3.100.184	0.2%	0.3%	0.2%	99.3%
Surfshark	194.169.169.169	0.0%	25.0%	25.0%	25.0%
Tiarap Public DNS	174.138.21.128	0.0%	0.0%	0.0%	100.0%
Verisign Public DNS	64.6.65.6	0.0%	0.0%	0.0%	99.9%
Yandex DNS	77.88.8.1	5.6%	4.7%	5.1%	84.7%

Appendix C.
Experiments of Public DNS Services, DNS Software, WIFI Routers, and Router OSes.

We conducted systematic tests on 43 well-known public DNS services, 8 recursive resolvers, 13 forwarders, 6 stub resolvers, 4 DNS libraries, 21 Wi-Fi routers, and 6 router OSes for 5 rounds. Due to the extensive set of 66 special characters, the corresponding results are presented vertically to accommodate the layout. Table 10 presents the support status of the public DNS services. The results for DNS software, Wi-Fi routers, and router operating systems are summarized in Table 11.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.

TABLE 10: Support for ASCII special characters in 43 public DNS services.

Vendor	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F	3A	3B	3C	3D	3E	3F	40	5B	5C	5D	5E	5F	60	7B	7C	7D	7E	7F
	SP†	!	”	#	$	%	&	’	(	)	*	+	,	-	.	/	:	;	<	=	>	?	@	[	\	]	^	_	‘	{	—	}	~		

TABLE 11: Support for ASCII special characters in 31 DNS software, 22 Wi-Fi routers, and 6 Wi-Fi router OSes.

Role	Software	SP†	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/	:	;	<	=	>	?	@	[	\	]	^	_	`	{	—	}	~
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F	3A	3B	3C	3D	3E	3F	40	5B	5C	5D	5E	5F	60	7B	7C	7D	7E	7F
Recursive	BIND	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Unbound	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Knot	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	PowerDNS	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Microsoft	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Simple DNS+	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Technitium	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
	MaraDNS	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Forwarder	Dnsmasq	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	CoreDNS	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	DNSDist	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	SmartDNS	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Pi-hole	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	pdnsd	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Acrylic DNS	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	AdGuard	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	AdGuard Home	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	DNS Safety	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Dual DHCP DNS	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	NxFilter	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	YogaDNS	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Stub	Linux	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Windows	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	MacOS	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	iOS	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Android	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	ChromeOS	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	-
Library	Python	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Golang	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	JavaScript	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

Appendix D.
Meta-Review

The following meta-review was prepared by the program committee for the 2026 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

D.1. Summary

The paper presents novel DNS vulnerabilities related to the mishandling of special characters in DNS resolvers, which facilitate previously-known DNS cache poisoning attacks. The poisoned caches can be on either the stub resolver, forwarder, or recursive resolver. The attack hinges on that common DNS software silently drops a query when it contains special characters. This increases the window during which an attacker can inject DNS responses (i.e., sends crafted responses with spoofed IP addresses), because the requester remains waiting for a response during that period. The attacker can then mount any of the previously established poisoning attacks, like Kaminsky and Tudoor, without having to race the legitimate responses. The paper also discusses how such silent dropping can be achieved to target some of the physical ANS replicas of a zone, thus manipulating the load-balancing algorithm, or all such replicas, thus causing a DoS. The paper tests susceptibility of common DNS software, and tests the attacks in practice through measurements studies.

D.2. Scientific Contributions

- Identifies an Impactful Vulnerability

D.3. Reasons for Acceptance

1) In spite of several editorial comments, the paper is generally well-written.
2) The identified vulnerabilities are critical and impactful.
3) Analysis is wide-scale and conclusive.
4) Authors responsibly disclosed the vulnerability to affected parties.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on September 30,2026 at 03:45:13 UTC from IEEE Xplore. Restrictions apply.